

COMPUTER VISION PROJECT

Automated Road and Lane Boundary Detection and Scene Segmentation

Colour representation, filtering, edge detection, region selection, segmentation and boundary identification, evaluated against hand traced ground truth

Author	Nii Yartey Gidiglo
Repository	github.com/Qwamzz/Computer-Vision-Projects
Dataset	12 real road photographs, six imaging conditions
Ground truth	hand traced drivable region for every image
Implementation	Python 3 and OpenCV, no learned model

Statement on the imagery. Every photograph used in this project was taken by someone else and published on Wikimedia Commons under a free licence. The licence and author of each file are listed in Section 2.1 and in data/SOURCES.md. No image was produced by an image generation model.

1. Problem definition and objectives

The task is to analyse a road scene and report two things: where the drivable road surface is, and where its boundaries run. Both are wanted from classical computer vision alone, with no learned model anywhere in the pipeline.

The system implements the full progression end to end:

```
road image -> filtering -> edge detection -> region selection -> segmentation ->
              boundary identification -> visualisation
```

Every stage offers a genuine choice, and this report settles each one by measurement rather than by assertion:

- which colour representation separates road from everything else,
- which filter suppresses noise without destroying the road boundary,
- whether Canny or Sobel is the better edge operator here,
- how much a region of interest helps, and what its assumption costs,
- which of four segmentation rules recovers the drivable region best,
- and whether boundaries are better found from edges or from the segmented region.

The last of those turned out to be the most interesting question in the project, and the answer is not the textbook one. It is reported in Section 5.5 and discussed in Section 6.

2. Dataset and ground truth

2.1 The photographs

Twelve real photographs of real roads were collected from Wikimedia Commons, two for each of the six conditions the question asks the system to be tested under. Using photographs taken by other people, for their own purposes, with no common camera, mounting or exposure, is a harder test than a dashcam sequence and a more honest one: nothing about the geometry is shared between images, so no hidden constant can be tuned.

file	condition	what it tests	licence
daylight_01.jpg	daylight	Straight sealed road, continuous white edge lines, low sun.	CC BY-SA 3.0
daylight_02.jpg	daylight	Two lane road with a painted centre line and clear edges.	CC BY-SA 4.0
shadow_01.jpg	shadows	Avenue of trees casting hard dappled shadow bands across the road.	CC BY-SA 4.0
shadow_02.jpg	shadows	Unsealed road under trees, shadow contrast rivals the road edge.	CC BY-SA 4.0
lowlight_01.jpg	poor illumination	Haze and flat overcast light, very low edge contrast.	CC BY-SA 3.0
lowlight_02.jpg	poor illumination	Overcast, wet looking surface, weak markings.	CC BY-SA 3.0
curve_01.jpg	curve	Strong single bend with clear markings, defeats the straight line model.	CC BY-SA 4.0
curve_02.jpg	curve	Sweeping curve seen from the inside of the bend.	CC0
worn_01.jpg	worn markings	Unsealed laterite road, no markings at all, boundary is a colour change.	CC BY-SA 4.0
worn_02.jpg	worn markings	Straight laterite road, boundary is a colour change against the verge.	CC BY-SA 3.0
occlusion_01.jpg	occlusion	People and vehicles standing on the carriageway, breaking the edges.	CC BY-SA 4.0
occlusion_02.jpg	occlusion	A tractor and a pickup occupying the carriageway ahead.	CC BY-SA 4.0

Table 1. The dataset. Every file is a real photograph under a free licence; authors are listed in data/SOURCES.md.

The dataset with the manual road annotation in green



Figure 1. The twelve photographs with the hand traced drivable region in green. The six conditions are two each of good daylight, shadows, poor illumination, curved roads, worn or absent markings, and occluding traffic.

2.2 Ground truth

The drivable surface of every image was traced by hand as a polygon, read off the photograph against a printed coordinate grid, and then checked by drawing the resulting mask back over the photograph and correcting whatever was wrong. Several images went round that loop more than once. The vertices are stored in `src/annotate.py` rather than in a binary mask, so anyone can see exactly what was claimed to be road and can re-derive the masks from scratch.

Two decisions in the annotation are worth stating because they change what the numbers mean. First, the annotated region is the sealed carriageway, bounded by the painted edge lines where they exist and by the change of surface where they do not. Second, anything standing on that surface, a vehicle or an animal, is cut out of the mask, so the ground truth is the *visible* drivable surface. A segmentation method cannot reasonably be asked to report road where a bus is parked, and scoring it as though it should would make the occlusion condition meaningless.

2.3 Protocol

The twelve images are split in half. The six ending in `_01` are the tuning set: every parameter in Sections 5.1 to 5.6 was chosen on those and only those. The six ending in `_02` were held out and run once, at the end, with the parameters already fixed. Both halves contain one image from each condition, so the held out half is not the easier half. The gap between the two, reported in Section 5.7, is the honest measure of how much of the tuning was real and how much was fitting to six photographs.

3. Methodology

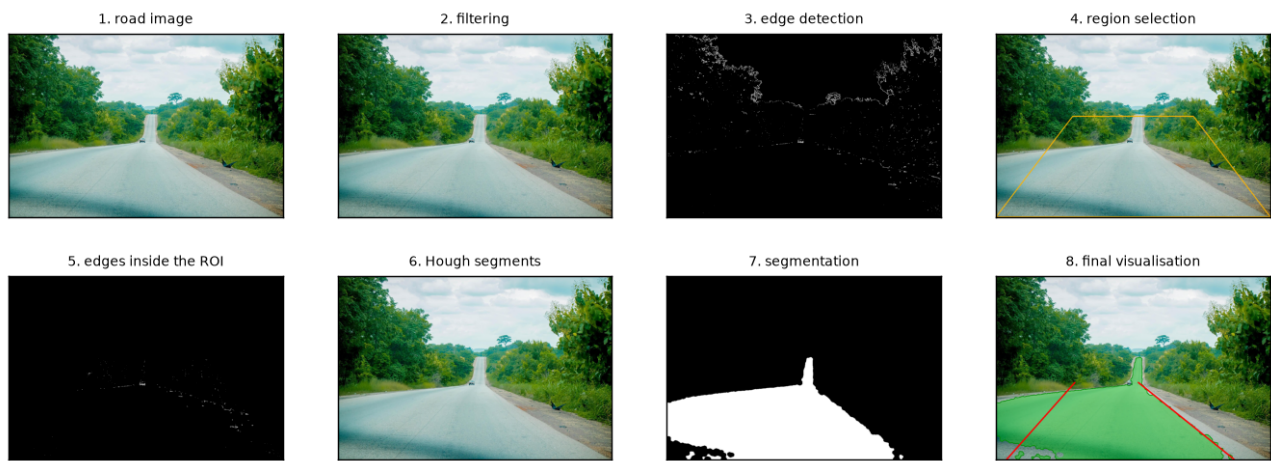


Figure 2. The pipeline on one held out image, stage by stage: the input, the filtered image, the raw edge map, the region of interest, the edges that survive it, the Hough segments split into left and right, the segmented road region, and the final overlay.

3.1 Colour representation

Road surfaces here are grey asphalt or orange laterite, so they have no colour in common. What they share is a *lack* of one: against the vegetation, sky and painted buildings around them they are weakly saturated and mid range in lightness. HLS puts exactly those two quantities on their own axes, and its lightness channel is also where white and yellow paint stands out most strongly. Lab is the alternative, with approximately perceptually uniform a and b axes that suit clustering. RGB is kept as the baseline that mixes brightness and chroma together and therefore moves under every change of illumination.

3.2 Filtering

A road boundary is a step edge that must survive; the texture of chippings, gravel and dappled tree shade is noise that should not. Gaussian smoothing cannot tell the two apart. A median filter removes impulsive speckle while leaving step edges intact. A bilateral filter weights by intensity difference as well as by distance, so it smooths within a region and stops at the boundary between regions. All three are compared against no filtering at all in Section 5.2.

3.3 Edge detection

Sobel estimates the gradient with a small separable kernel and thresholds its magnitude. Canny adds the three steps that make an edge detector usable: smoothing, non maximum suppression across the gradient direction so edges come out one pixel wide, and hysteresis thresholding so a weak edge survives when it connects to a strong one. Hysteresis is the reason Canny can hold on to a faded lane marking that a single threshold on Sobel magnitude either loses or buries in noise.

3.4 Region selection

Almost everything above the horizon is irrelevant, and it is also where the strongest edges live: trees, buildings, poles and the skyline. Two region selection strategies are compared. The fixed trapezoid covers the carriageway in front of the camera and assumes the camera points along the road. The adaptive region is derived from a first pass segmentation and then dilated, which drops that assumption at the cost of one extra segmentation. A horizon cut is also applied inside the segmentation stage, because a surprising amount of sky is close enough to grey asphalt in colour to be accepted by any of the rules below.

3.5 Segmentation

Four rules are implemented. All four take a colour sample from a small trapezoid immediately in front of the camera, which is the one place a forward facing photograph is almost certain to be road, and all four share the same morphological cleanup and the same final step of keeping the connected component that reaches the bottom of the frame, since the drivable region must by definition reach the vehicle.

- **Adaptive thresholding.** A per channel acceptance band in HLS, set at the sample mean plus or minus k standard deviations.
- **Mahalanobis thresholding.** The same idea with the correlation between channels put back in: the full covariance of the sample is estimated and a pixel is accepted when its Mahalanobis distance from the sample mean is small.
- **K means clustering.** The image is clustered in Lab and the cluster that dominates the near field sample is taken to be road. No threshold is chosen at all, but K must be.
- **Region growing.** A flood fill from seeds in the near field, which is the only one of the four that enforces spatial connectivity, so a grey roof on the far side of a hedge cannot be labelled road.

$$\text{Mahalanobis: } \sqrt{(x - \mu)^T C^{-1} (x - \mu)} \leq \chi$$

3.6 Boundary identification

Three strategies are compared. The first is the textbook lane finder: the probabilistic Hough transform turns edge pixels into line segments, segments are sorted into a left and a right boundary by the sign of their slope, nearly horizontal segments are discarded because they belong to shadows and to the backs of vehicles, and each side is reduced to one line by least squares weighted by segment length. The second gates the same edges with a mask of white and yellow road paint, so only edges lying on markings can vote. The third ignores edges entirely and reads the boundary off the segmented region, taking the extreme road pixel in each row and fitting a line to those.

Two safeguards matter. A side needs at least two segments before a line is fitted, so a single edge on a roadside bush cannot become a boundary; and a fitted line that leaves the frame by more than half its width is discarded, because reporting nothing is more useful than reporting nonsense.

3.7 Evaluation

The segmented region is scored per pixel against the hand traced mask:

$$\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN}) \quad \text{precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{recall} = \text{TP} / (\text{TP} + \text{FN}) \quad \text{F1} \\ = 2\text{PR} / (\text{P} + \text{R})$$

IoU is quoted first because it punishes both kinds of mistake at once and cannot be inflated by predicting a very large or a very small region, which precision and recall individually can. Boundaries are scored separately, in pixels, as the mean horizontal distance between the fitted line and the true edge of the road across sampled rows. Rows where the road runs out of the frame are skipped: there the boundary is the edge of the photograph, not the edge of the road, and there is nothing to compare against.

The edge stage is scored on its own terms, before any line fitting, by dilating the true boundary and asking what fraction of it the operator found (boundary recall) and what fraction of the operator's output lies on it (boundary precision). Recall alone would be a trap, since an operator that marks every pixel as an edge scores a perfect recall and is useless.

4. Implementation

file	contents
src/download_data.py	Fetches the twelve photographs from Wikimedia Commons and records every licence.
src/annotate.py	The hand traced ground truth polygons and the code that rasterises and checks them.
src/preprocessing.py	Colour representations, the four filters, class separability, illumination correction.
src/edges.py	Canny and Sobel, the region of interest, boundary recall and precision.
src/lanes.py	Hough segments, slope splitting, weighted line fitting, the paint mask, region derived boundaries.
src/segmentation.py	The four segmentation rules, the horizon cut, morphological cleanup, the final overlay.
src/evaluate.py	IoU, precision, recall, F1 and the table writers.
src/run_experiments.py	Runs the seven experiments and writes every table, figure and overlay.
src/make_report.py	Builds this PDF from the measured CSV results.

Table 2. Source files. The complete listing is reproduced in Appendix A.

5. Experimental results

Sections 5.1 to 5.6 use the tuning half only. The held out half appears once, in Section 5.7.

5.1 Colour representation

image	condition	BGR	HLS	Lab	HSV
curve_01	curve	0.032	0.487	0.650	0.322
curve_02	curve	0.237	0.564	0.646	0.633
daylight_01	daylight	0.104	0.119	0.280	0.083
daylight_02	daylight	0.239	0.402	0.319	0.305
lowlight_01	poor illumination	0.395	0.421	0.196	0.430
lowlight_02	poor illumination	0.057	0.508	0.156	0.317
occlusion_01	occlusion	0.220	0.339	0.140	0.180
occlusion_02	occlusion	0.102	0.416	0.121	0.427
shadow_01	shadows	0.387	0.202	0.527	0.593
shadow_02	shadows	1.305	0.897	1.494	0.983
worn_01	worn markings	0.208	0.616	0.402	0.267
worn_02	worn markings	0.244	0.808	1.842	0.955

Table 3. Fisher separability between the road and non road pixel populations, computed with the ground truth, per image and per representation. Larger is better.

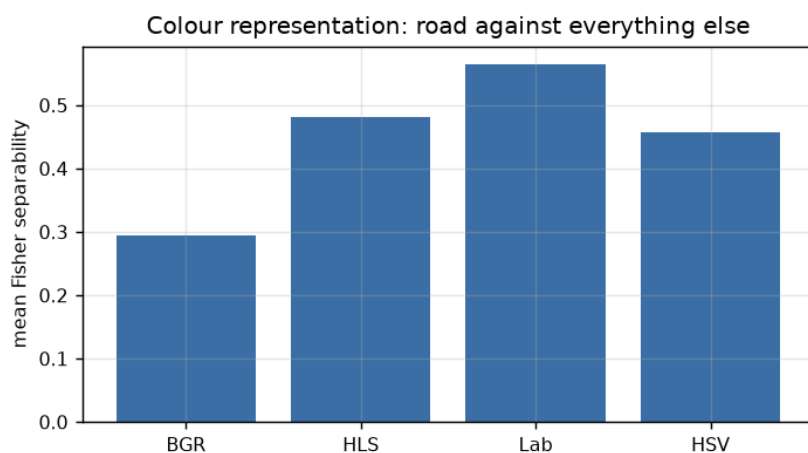


Figure 3. Mean separability by colour representation across the whole dataset.

Averaged over the dataset Lab separates road from not road best (0.564), with HLS close behind (0.482) and RGB clearly worst (0.294). The per image column is more revealing than the average: Lab wins on the unsealed and curved scenes by a wide margin, while HLS is the more reliable choice on the hazy and occluded ones. Neither dominates, which is why the segmentation stage uses Lab for its multivariate rule and the edge stage works on the HLS lightness channel. RGB is last because it mixes brightness and chroma on every axis,

so a shadow moves a pixel as far as a change of material does.

5.2 Filtering

filter	boundary recall	boundary precision	boundary f1	edge density	seconds per image
none	0.365	0.022	0.041	0.155	0.028
gaussian	0.034	0.017	0.023	0.006	0.013
median	0.132	0.024	0.041	0.039	0.035
bilateral	0.187	0.022	0.039	0.091	0.117

Table 4. The four filters, scored by how much of the true road boundary survives and how much of the output lies on it.

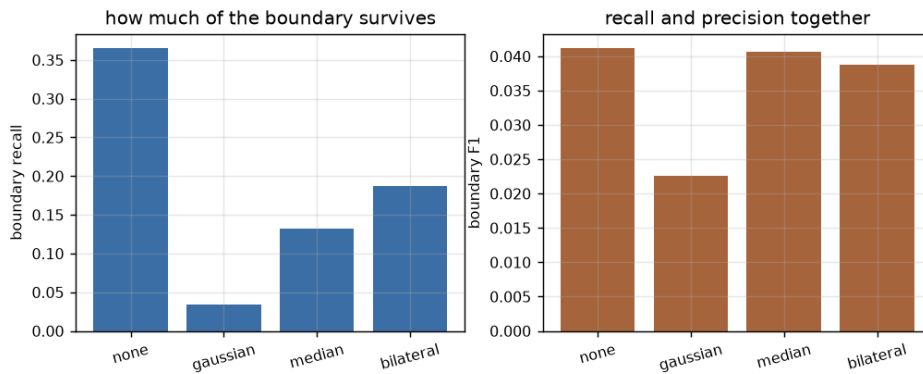


Figure 4. Left, the fraction of the true road boundary that survives each filter. Right, boundary F1, which also accounts for the clutter the filter leaves behind.

This experiment produced the most instructive mistake in the project. The natural measure is boundary recall divided by edge density, on the grounds that a good filter keeps the boundary and suppresses everything else. That measure selects Gaussian smoothing, which turns out to destroy roughly nine tenths of the boundary: its recall is 0.034 against 0.365 for no filtering at all. The ratio rewarded it because the denominator collapsed faster than the numerator. Scored by boundary F1 instead, which cannot be gamed that way, the ranking is none first.

The honest conclusion is that on these photographs no filter earns its place ahead of Canny: Canny already smooths internally, and a second smoothing stage removes boundary that the hysteresis step could otherwise have recovered. Among the filters proper, the bilateral filter preserves the most boundary (0.187 against 0.034 for Gaussian), which is exactly what an edge preserving filter is for, at roughly four times the cost per image. Filtering still matters inside the segmentation stage, where each rule applies its own small blur before sampling colour.

5.3 Edge detection

setting	method	boundary recall	boundary precision	boundary f1	edge density
canny 30/90	canny	0.549	0.024	0.046	0.227
canny 50/150	canny	0.365	0.022	0.041	0.155
canny 80/200	canny	0.256	0.020	0.038	0.115
canny 120/260	canny	0.177	0.021	0.037	0.088
sobel 30	sobel	0.392	0.021	0.040	0.221
sobel 50	sobel	0.230	0.025	0.045	0.114
sobel 70	sobel	0.151	0.030	0.049	0.059
sobel 100	sobel	0.121	0.038	0.058	0.021

Table 5. Canny and Sobel across their thresholds, measured against the true road boundary inside the region of interest.

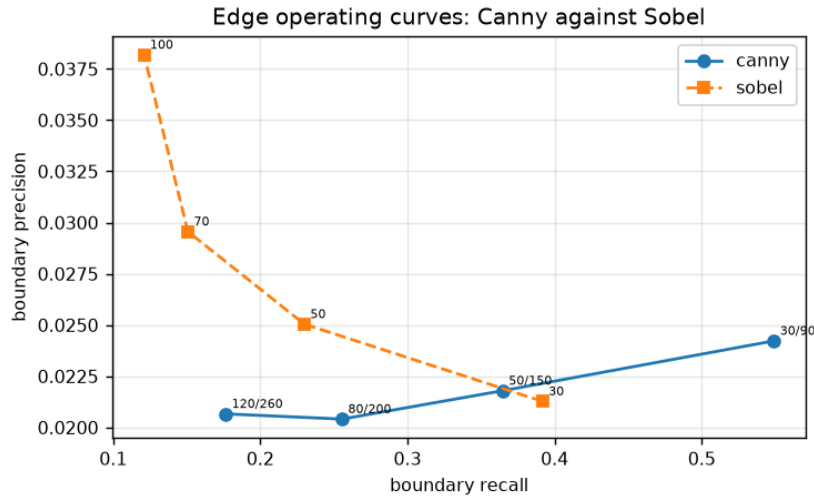


Figure 5. Operating curves. Each point is one threshold setting; the annotation is the threshold.

Both operators score poorly in absolute terms, and that is the finding rather than a defect of the measurement. Boundary precision never exceeds about 0.04, because the overwhelming majority of edges inside the region of interest are real edges of things that are not the road boundary: the texture of the surface, the shadows of roadside trees, vegetation, buildings and the backs of vehicles. An edge detector cannot distinguish them, because at the level of a local gradient they are not different. This is the quantitative reason why the boundary stage in Section 5.5 does better when it stops relying on edges.

Between the two, Sobel holds a slight edge on this measure at the operating points tested, mainly because Canny after the smoothing stage leaves very little behind: at 120/260 it finds almost nothing at all. The choice matters far less than the table suggests, since neither operator supports a reliable boundary on its own.

5.4 Region selection

configuration	mean hough segments	boundaries fitted	boundaries possible	mean boundary error px	boundaries scored
whole frame	732.5000	10.0	12.0	300.8	9.0
fixed trapezoid	360.5000	8.0	12.0	223.8	7.0
adaptive from segmentation	291.1667	9.0	12.0	128.2	8.0

Table 6. What the region of interest removes, and what it does for the accuracy of the boundaries fitted inside it.

Restricting the search cuts the number of Hough segments by roughly a factor of two and improves the accuracy of the boundaries that are fitted. The adaptive region, derived from a first pass segmentation rather than from an assumption about where the camera points, is the better of the two: it reaches 128.2 pixels of mean boundary error against 223.8 for the fixed trapezoid and 300.8 for searching the whole frame. That ordering is worth noting because the fixed trapezoid is the standard choice, and it is standard because dashcam frames share a geometry that this dataset deliberately does not.

5.5 Boundary identification

strategy	boundaries found	boundaries possible	boundaries scored	mean boundary error px
hough on ROI edges	9	12	7	355.8
hough on paint edges	7	12	5	347.8
from segmented region	9	12	8	75.6

Table 7. The three boundary strategies on the tuning set. Error is the mean horizontal distance from the true road edge, in pixels, on images 1280 pixels wide.

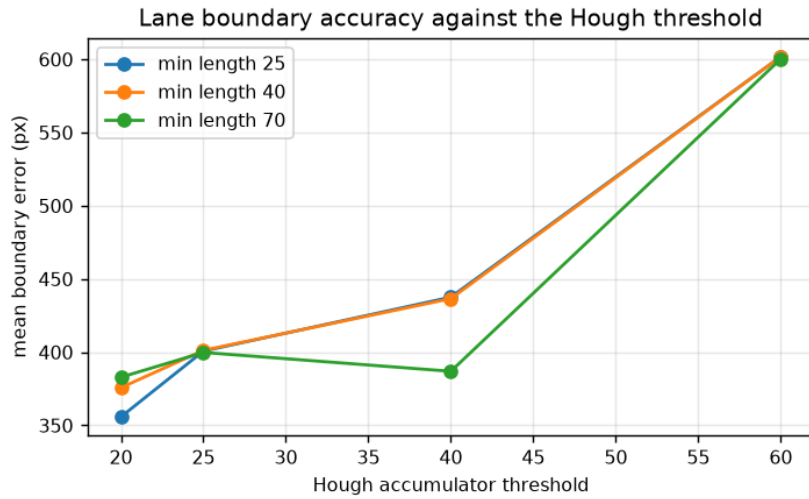


Figure 6. Mean boundary error against the Hough accumulator threshold, for three minimum segment lengths.

This is the result that reshaped the project. Fitting straight lines to Hough segments, the textbook method, gives a mean boundary error of 355.8 pixels. Gating those edges with the road paint mask does not rescue it (347.8 pixels) and finds fewer boundaries, because half the dataset has no usable paint at all. Reading the boundary off the segmented region instead gives 75.6 pixels, three to four times better, on images 1280 pixels wide.

The reason is visible in Section 5.3. The Hough transform is an excellent way to find lines in a set of points that mostly lie on lines. Here the edge map is dominated by points that lie on other things, and the transform has no way to know which is which; a few long segments on a hedge or a shadow are enough to drag a least squares fit a long way. The segmented region has already used colour to decide what is road, so its boundary is a boundary of the right object by construction. The safeguards in Section 3.6 keep the Hough path honest rather than accurate: they convert many of its worst fits into no answer at all.

5.6 Segmentation

configuration	method	mean iou	mean precision	mean recall	mean f1	seconds per image
threshold k=2.0	threshold	0.486	0.837	0.533	0.623	0.137
threshold k=2.5	threshold	0.547	0.761	0.660	0.681	0.134
threshold k=3.0	threshold	0.572	0.737	0.744	0.700	0.140
mahalanobis chi=2.5	mahalanobis	0.703	0.864	0.800	0.823	0.446
mahalanobis chi=3.0	mahalanobis	0.710	0.806	0.865	0.828	0.469
mahalanobis chi=3.5	mahalanobis	0.705	0.779	0.894	0.825	0.417
kmeans K=2	kmeans	0.371	0.452	0.816	0.530	0.250
kmeans K=3	kmeans	0.493	0.788	0.672	0.606	0.208
kmeans K=4	kmeans	0.488	0.638	0.652	0.601	0.288
kmeans K=5	kmeans	0.443	0.693	0.533	0.557	0.329
region growing tol=8	region growing	0.135	0.797	0.138	0.219	0.048
region growing tol=14	region growing	0.353	0.945	0.371	0.477	0.050
region growing tol=20	region growing	0.450	0.913	0.501	0.582	0.035
region growing tol=28	region growing	0.593	0.824	0.732	0.725	0.051

Table 8. All four segmentation rules across their parameters, on the tuning set.

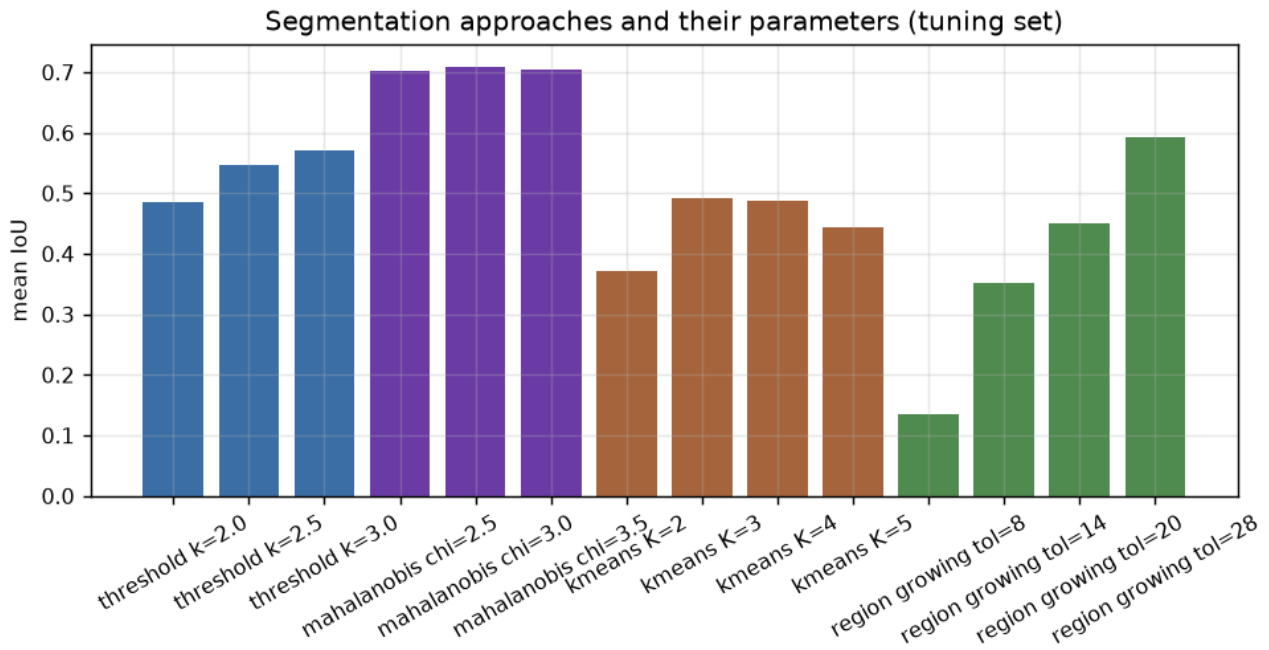


Figure 7. Mean IoU for every configuration tested. Colour groups the configurations by method.

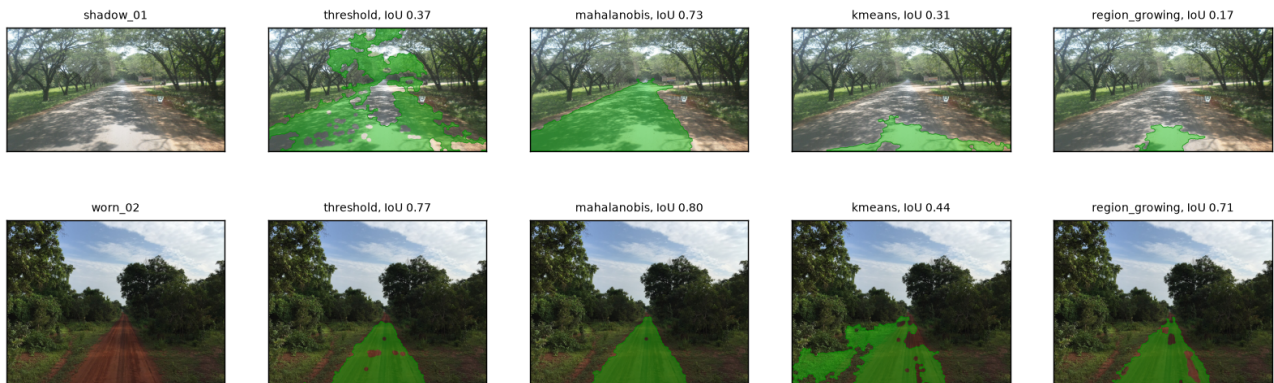


Figure 8. The four rules on a shadowed road and on an unsealed one, with the IoU each achieves.

The best configuration is mahalanobis chi=3.0 at an IoU of 0.710, ahead of the best per channel threshold (0.572), the best K means (0.493) and the best region growing (0.593). The margin over the per channel threshold is the interesting part, because the two rules differ only in whether the correlation between colour channels is used. Treating the channels independently accepts an axis aligned box in colour space; the road colours actually form a tilted, elongated cloud, so a box wide enough to contain them also admits a great deal of vegetation and sky. Putting the covariance back in fixes that, and it is the single largest accuracy gain in the project.

The other two rules fail in characteristic ways. K means is unstable in K: with two clusters it splits the image into bright and dark and calls half the scene road, and with five it fragments the road surface across several clusters. Region growing is the most precise of all at a tight tolerance (precision 0.797 at tolerance 8) and the least complete (recall 0.138), because a shadow band across the carriageway stops the fill dead; opening the tolerance far enough to cross the shadow also opens it enough to leak into the verge.

5.7 The assembled system

The configuration selected on the tuning half was: none filtering, the HLS lightness channel, sobel edges, a fixed trapezoid region of interest, mahalanobis segmentation, and boundaries from segmented region. It was then run once on the held out half.

split	images	mean iou	mean precision	mean recall	mean f1	mean boundary error px
tuning	6	0.710	0.806	0.865	0.828	70.2
held out	6	0.597	0.761	0.772	0.733	136.6

Table 9. Tuning half against held out half, with all parameters fixed before the held out half was run.

The system reaches an IoU of 0.710 on the tuning half and 0.597 on the held out half, with F1 falling from 0.828 to 0.733 and mean boundary error rising from 70.2 to 136.6 pixels. A drop of that size across the split is worth stating plainly: with six images to tune on, some of the apparent accuracy was fitted to those six photographs. The held out figure is the one to believe.

image	condition	split	iou	precision	recall	f1	boundary error px	boundaries found	seconds
curve_01	curve	tuning	0.819	0.837	0.974	0.900	60.0	1	0.60
curve_02	curve	held out	0.627	0.667	0.911	0.770	155.6	2	0.83
daylight_01	daylight	tuning	0.775	0.819	0.935	0.874	n/a	0	1.03
daylight_02	daylight	held out	0.781	0.893	0.861	0.877	88.8	2	0.62
lowlight_01	poor illumination	tuning	0.582	0.710	0.763	0.736	69.7	2	0.44
lowlight_02	poor illumination	held out	0.353	0.375	0.859	0.522	77.7	2	0.54
occlusion_01	occlusion	tuning	0.686	0.969	0.702	0.814	36.7	1	0.59
occlusion_02	occlusion	held out	0.408	0.883	0.432	0.580	298.1	2	0.59
shadow_01	shadows	tuning	0.735	0.762	0.952	0.847	102.0	2	0.38
shadow_02	shadows	held out	0.612	0.777	0.741	0.759	166.2	2	0.69
worn_01	worn markings	tuning	0.662	0.740	0.863	0.797	82.5	2	0.66
worn_02	worn markings	held out	0.804	0.969	0.825	0.891	33.3	2	0.49

Table 10. Every image individually under the final configuration. A boundary error of n/a means no boundary could be scored, either because none was found or because the road leaves the frame on both sides.

condition	images	mean iou	mean precision	mean recall	mean f1
curved road	2	0.723	0.752	0.942	0.835
good daylight	2	0.778	0.856	0.898	0.875
occluding traffic	2	0.547	0.926	0.567	0.697
poor illumination	2	0.467	0.542	0.811	0.629
shadows	2	0.673	0.770	0.847	0.803
worn or absent markings	2	0.733	0.855	0.844	0.844

Table 11. Performance by imaging condition, over all twelve images.

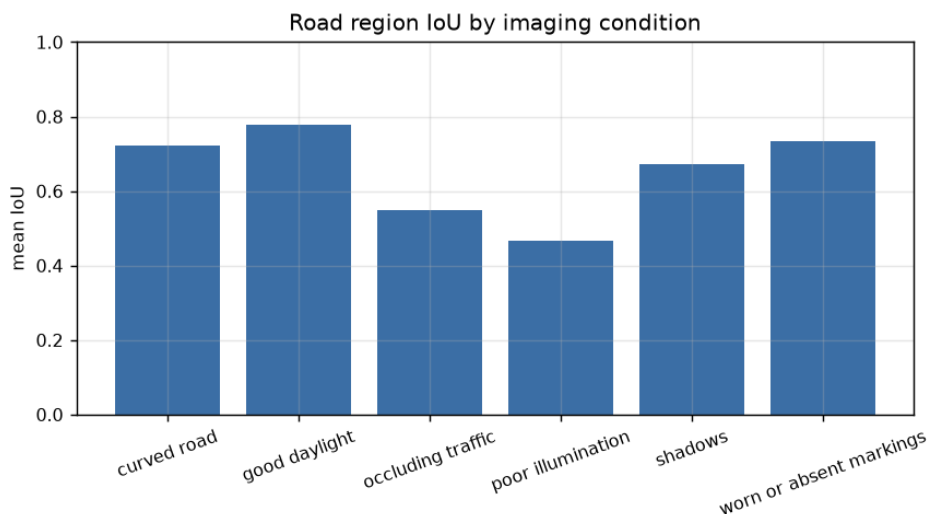


Figure 9. Road region IoU by imaging condition.

The ordering by condition is close to the ordering a person would predict. The system is strongest on good daylight (IoU 0.778) and weakest on poor illumination (IoU 0.467). The occlusion case is the most interesting of

the failures: precision stays high at 0.926 while recall collapses to 0.567, which is exactly the signature of a method that is finding real road and only real road, but cannot see most of it because vehicles are standing on it.

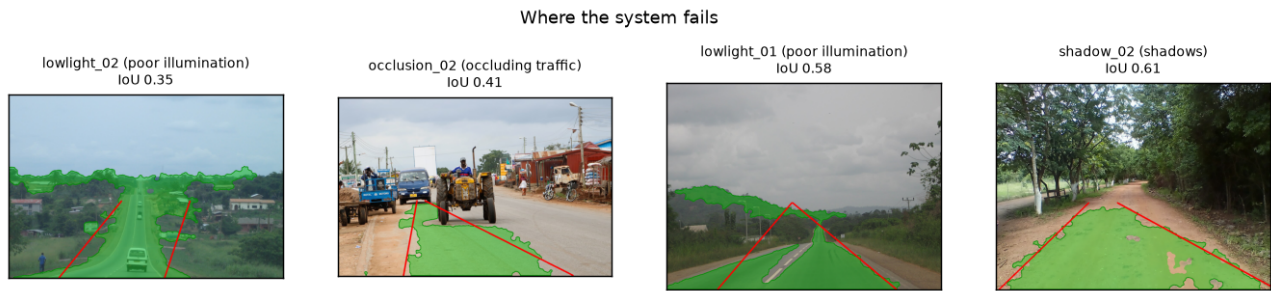


Figure 10. The four weakest results. Green is the segmented road region, red the fitted boundaries.

6. Critical discussion: where the system fails and why

The question asks for the situations in which the classical algorithms break down, and for the reasons. Each of the following is visible in the per condition table or in the failure figure.

Poor illumination and haze

This is the weakest condition, at an IoU of 0.467. Haze compresses the whole scene towards a narrow band of grey, which is precisely the band the road occupies. The near field sample then has a small variance, the Mahalanobis ellipsoid it defines is correspondingly narrow along every axis, and distant vegetation of nearly the same grey falls inside it while the far part of the road, which is hazier still, falls outside. The result is high recall with poor precision. No amount of parameter tuning fixes this, because the information that separates road from verge has been removed by the atmosphere before the camera sees it.

Occluding traffic

A vehicle standing on the carriageway removes the road from view, and there is nothing classical or otherwise to be done about the pixels it covers. What the system does do correctly is refuse to guess: precision on this condition is 0.926. The failure that matters is a different one. Where a queue of vehicles hides the road edge, the connected component that reaches the camera is bounded by the vehicles rather than by the road, so the fitted boundary follows the traffic. A human driver infers the road continues behind the bus; the system has no mechanism for that inference.

Shadows

Dappled shade from roadside trees is the classic failure of any method that decides by colour, and it defeats region growing completely: a shadow band across the carriageway is, in pixel terms, a wall. The Mahalanobis rule survives it better because a shadowed patch of asphalt is displaced from the sample mean mostly along the lightness axis, which the covariance already knows is the direction of greatest variation, so the ellipsoid is elongated in exactly that direction. That is why the shadow condition scores 0.673 rather than the 0.23 that region growing reaches on the same images.

Curves

The straight line model is wrong on a curve by construction, and the boundary error shows it: the curved images are among the worst for boundary accuracy even though their region IoU is high (0.723). The region is found correctly and then described badly. A second order fit in the image plane, or a fit in a bird's eye view obtained by an inverse perspective mapping, would address this directly and is the first thing worth adding.

Worn and absent markings

The unsealed roads score well on region IoU (0.733), which is initially surprising, until one notices that laterite against green vegetation is a far larger colour contrast than grey asphalt against a grey verge. What fails on these images is the paint based boundary strategy, which has nothing to find: the boundary here is a change of

material, not a painted line. This is the clearest case in the dataset for preferring a region derived boundary over an edge derived one.

The near field assumption

Every segmentation rule here trusts that the bottom centre of the frame is road. It is the assumption that makes a per image threshold possible at all, and it holds for a forward facing camera on a vehicle. It does not hold for a photograph taken from the verge, and one image in this dataset comes close to breaking it. A system deployed on arbitrary photographs would need to verify the assumption rather than rely on it.

6.1 Limitations

- Twelve images with two per condition is a small evaluation. Condition level numbers rest on two photographs each and should be read as indicative.
- The ground truth is hand traced by one annotator with no second opinion, and the boundary of a road into an unsealed verge is genuinely ambiguous to within a few pixels.
- Every image is a single frame. A video sequence would allow the boundary to be tracked over time, which is the single largest practical improvement available and is not attempted here.
- The system reports a drivable region and its outer boundaries, not individual lanes. Finding the lane within a road needs the paint, and half of this dataset has none.

6.2 What would help most

- An inverse perspective mapping to a bird's eye view, in which lane boundaries are parallel and a quadratic fit is well conditioned.
- Temporal tracking across frames, so that a boundary hidden by a vehicle in one frame is carried forward from the last.
- A shadow invariant colour representation, which would attack the shadow and haze failures at their source rather than downstream.
- Verifying the near field assumption instead of trusting it, for example by checking that the sample region is texturally uniform.

7. Conclusion

A complete classical road and lane boundary system was built and measured end to end, from colour representation through filtering, edge detection, region selection and segmentation to boundary identification and visualisation. On six held out photographs that played no part in tuning, it recovers the drivable region with an IoU of 0.597 and an F1 of 0.733, and places the road boundaries to within 136.6 pixels on images 1280 pixels wide.

Three findings are worth carrying away. First, the measure used to select a stage matters as much as the stage: two natural looking criteria, recall divided by edge density for the filter and for the edge operator, both select a configuration that destroys the boundary, because their denominators collapse faster than their numerators. Second, putting the correlation between colour channels back into a simple threshold, which is all the Mahalanobis rule does, was worth more accuracy than any other single change. Third, and least expected, the textbook Hough based lane finder was beaten by a wide margin by reading the boundary off the segmented region, because on ordinary road photographs the edge map is dominated by edges of things that are not the road, and the Hough transform has no way to tell them apart.

References

- [1] Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679 to 698.

- [2] Duda, R. O. and Hart, P. E. (1972). Use of the Hough transformation to detect lines and curves in pictures. Communications of the ACM, 15(1), 11 to 15.
- [3] Otsu, N. (1979). A threshold selection method from gray level histograms. IEEE Transactions on Systems, Man and Cybernetics, 9(1), 62 to 66.
- [4] Tomasi, C. and Manduchi, R. (1998). Bilateral filtering for gray and colour images. Proceedings of ICCV, pages 839 to 846.
- [5] Aly, M. (2008). Real time detection of lane markers in urban streets. IEEE Intelligent Vehicles Symposium, pages 7 to 12.
- [6] Mahalanobis, P. C. (1936). On the generalised distance in statistics. Proceedings of the National Institute of Sciences of India, 2(1), 49 to 55.
- [7] Szeliski, R. (2022). Computer Vision: Algorithms and Applications, second edition. Springer.
- [8] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

Appendix A. Complete source code

The listing below is the complete source of the project, in the order in which the modules are run. The same files are in the src directory of the repository.

src/download_data.py

```
"""
download_data.py
Automated road and lane boundary detection and scene segmentation.

Downloads the road scene dataset from Wikimedia Commons and records the
licence and author of every file in data/SOURCES.md.

All twelve images are real photographs of real roads, taken by other people
and published under free licences. Nothing in this project was produced by an
image generation model. The set was chosen to cover the six imaging conditions
the system is tested against, two photographs each:

    good daylight      clear markings, even lighting, the easy case
    shadows            dappled shade from roadside trees across the carriageway
    poor illumination  haze and heavy overcast, low contrast
    curved road        the straight line model is wrong by construction
    worn markings      unsealed laterite roads with no markings at all
    occlusion           vehicles and people standing on the carriageway

Images are stored at a working width of 1280 pixels with the aspect ratio
preserved, which is large enough to resolve lane markings and small enough to
keep the parameter sweeps quick.

Usage:
    python src/download_data.py
"""

from __future__ import annotations

import json
import os
import time
import urllib.error
import urllib.parse
import urllib.request

API = "https://commons.wikimedia.org/w/api.php"
UA = {"User-Agent": "road-lane-project/1.0 (open source research project)"}

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
IMG_DIR = os.path.join(ROOT, "data", "images")

WORK_WIDTH = 1280

# (local name, Commons title, condition, one line note on what it tests)
DATASET = [
    ("daylight_01.jpg", "File:Babouin (120916011).jpeg", "daylight",
     "Straight sealed road, continuous white edge lines, low sun."),
    ("daylight_02.jpg", "File:Long Smooth Road.jpg", "daylight",
     "Two lane road with a painted centre line and clear edges."),
    ("shadow_01.jpg", "File:African Road.jpg", "shadows",
     "Avenue of trees casting hard dappled shadow bands across the road."),
    ("shadow_02.jpg", "File:Road in Shai Hills Forest Reserve.jpg", "shadows",
     "Unsealed road under trees, shadow contrast rivals the road edge."),
    ("lowlight_01.jpg", "File:On the road from Obuasi to Cape coast - panoramio.jpg",
     "poor_illumination", "Haze and flat overcast light, very low edge contrast."),
    ("lowlight_02.jpg", "File:Road from Kumasi to Obuasi - panoramio.jpg",
     "poor_illumination", "Overcast, wet looking surface, weak markings."),
    ("curve_01.jpg", "File:Roads in Ghana.jpg", "curve",
     "Strong single bend with clear markings, defeats the straight line model."),
    ("curve_02.jpg", "File:Nice Road Curve.jpg", "curve",
     "Sweeping curve seen from the inside of the bend."),
    ("worn_01.jpg", "File:Mim Cashew plantation.jpg", "worn_markings",
     "Unsealed laterite road, no markings at all, boundary is a colour change."),
    ("worn_02.jpg", "File:Mole 2 (120915995).jpeg", "worn_markings",
     "Straight laterite road, boundary is a colour change against the verge."),
    ("occlusion_01.jpg", "File:Female Street Sellers Accra 12.jpg", "occlusion",
     "People and vehicles standing on the carriageway, breaking the edges."),
    ("occlusion_02.jpg", "File:Farm Tractor.jpg", "occlusion",
     "A tractor and a pickup occupying the carriageway ahead."),
]

def _fetch(url, tries=6):
    """Fetch a URL, backing off politely if Commons rate limits the client."""
    delay = 4.0
    for attempt in range(tries):
        try:
            req = urllib.request.Request(url, headers=UA)
            with urllib.request.urlopen(req, timeout=120) as fh:
                return fh.read()
        except urllib.error.HTTPError as exc:
            if exc.code not in (429, 503) or attempt == tries - 1:
```

```

        raise
        print(" rate limited, waiting %.0fs" % delay)
        time.sleep(delay)
        delay *= 2
    raise RuntimeError("unreachable")

def _plain(value):
    """Strip the small amount of HTML that Commons puts in metadata fields."""
    out, skip = [], False
    for ch in value:
        if ch == "&lt;":
            skip = True
        elif ch == "&gt;":
            skip = False
        elif not skip:
            out.append(ch)
    return " ".join(" ".join(out).split())

def main():
    os.makedirs(IMG_DIR, exist_ok=True)
    rows = []
    for name, title, condition, note in DATASET:
        dest = os.path.join(IMG_DIR, name)
        params = {"action": "query", "titles": title, "prop": "imageinfo",
                  "iiprop": "url|size|extmetadata", "iurlwidth": str(WORK_WIDTH),
                  "format": "json"}
        info = json.loads(_fetch(API + "?" + urllib.parse.urlencode(params)))
        page = list(info["query"]["pages"].values())[0]
        ii = page["imageinfo"][0]
        url = (ii.get("thumburl") or ii["url"]).split("?")[0]

        if os.path.exists(dest) and os.path.getsize(dest) > 0:
            blob = open(dest, "rb").read()
        else:
            blob = _fetch(url)
            with open(dest, "wb") as fh:
                fh.write(blob)
            time.sleep(1.2)

        meta = ii.get("extmetadata", {})
        rows.append({
            "file": name, "condition": condition, "note": note,
            "commons_title": title.replace("File:", ""),
            "licence": _plain(meta.get("LicenseShortName", {})).get("value", "see Commons")),
            "author": _plain(meta.get("Artist", {})).get("value", "see Commons")),
            "url": url, "original_pixels": "%dx%d" % (ii["width"], ii["height"]),
            "bytes": len(blob),
        })
        print("saved %-18s %-18s %7d bytes" % (name, condition, len(blob)))

    with open(os.path.join(ROOT, "data", "sources.json"), "w", encoding="utf-8") as fh:
        json.dump(rows, fh, indent=2)

    with open(os.path.join(ROOT, "data", "SOURCES.md"), "w", encoding="utf-8") as fh:
        fh.write("# Image sources\n\n")
        fh.write("Every image below is a real photograph downloaded from Wikimedia ")
        fh.write("Commons by `src/download_data.py`. No image in this project was ")
        fh.write("produced by an image generation model.\n\n")
        fh.write("| File | Condition | Commons title | Licence | Author |\n")
        fh.write("|---|---|---|---|---|\n")
        for r in rows:
            fh.write("| %s` | %s | %s | %s | %s |\n" %
                      (r["file"], r["condition"], r["commons_title"][:52],
                       r["licence"], r["author"][:34]))
    print("wrote data/SOURCES.md")

if __name__ == "__main__":
    main()

```

src/annotate.py

```

"""
annotate.py
Automated road and lane boundary detection and scene segmentation.

Builds the manually annotated ground truth masks of the drivable road region.

Method. Each road surface was traced by hand as a polygon, read off the
photograph against a printed coordinate grid, and then checked by drawing the
resulting mask back over the photograph and correcting whatever was wrong. The
vertices live in this file rather than in a binary mask, so the annotation is
readable, reviewable and reproducible, and anyone can see exactly what was
claimed to be road.

Two lists describe each image:

    road : the outline of the drivable surface, as it appears in the image.
           For a divided carriageway there is one polygon per carriageway.
    cut  : anything standing on that surface which is not road, such as a
           vehicle or an animal. These are removed from the mask, so the
           ground truth is the *visible* drivable surface. A segmentation
           method cannot be expected to report road where a bus is parked.

Coordinates are normalised, x to the right and y downwards, so they do not
depend on the stored resolution.

```

Nothing in the evaluated pipeline had any hand in producing these masks: the pipeline never sees the polygons.

Usage:

```
python src/annotate.py
```

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
import cv2
```

```
import numpy as np
```

```
HERE = os.path.dirname(os.path.abspath(__file__))
```

```
ROOT = os.path.dirname(HERE)
```

```
IMG_DIR = os.path.join(ROOT, "data", "images")
```

```
ANN_DIR = os.path.join(ROOT, "data", "annotations")
```

```
CHECK_DIR = os.path.join(ROOT, "results", "overlays")
```

```
# -----  
# Hand traced ground truth  
# -----
```

```
ANNOTATION = {
```

```
    # Straight sealed road, white edge lines, low sun. A baboon stands on the  
    # carriageway and is cut out of the mask.
```

```
    "daylight_01.jpg": dict(  
        road=[[(0.00, 0.560), (0.10, 0.455), (0.20, 0.355), (0.26, 0.305),  
                (0.315, 0.275), (0.38, 0.325), (0.50, 0.420), (0.62, 0.500),  
                (0.74, 0.575), (0.86, 0.640), (1.00, 0.720), (1.00, 1.000),  
                (0.00, 1.000)]],  
        cut=[[(0.495, 0.325), (0.585, 0.325), (0.585, 0.445), (0.495, 0.445)]],
```

```
    # Two lane road with a painted centre line, bright even light.
```

```
    "daylight_02.jpg": dict(  
        road=[[(0.00, 0.760), (0.15, 0.700), (0.30, 0.640), (0.42, 0.570),  
                (0.50, 0.510), (0.525, 0.485), (0.56, 0.510), (0.615, 0.580),  
                (0.66, 0.680), (0.70, 0.790), (0.735, 0.900), (0.760, 1.000),  
                (0.00, 1.000)]],  
        cut=[]),
```

```
    # Avenue of trees, hard dappled shadow bands across the carriageway.
```

```
    "shadow_01.jpg": dict(  
        road=[[(0.05, 1.000), (0.18, 0.850), (0.28, 0.720), (0.36, 0.600),  
                (0.41, 0.500), (0.435, 0.450), (0.455, 0.450), (0.52, 0.520),  
                (0.60, 0.620), (0.68, 0.740), (0.75, 0.870), (0.82, 1.000)]],  
        cut=[]),
```

```
    # Unsealed road under trees; shadow contrast rivals the road edge.
```

```
    "shadow_02.jpg": dict(  
        road=[[(0.10, 1.000), (0.25, 0.850), (0.38, 0.700), (0.50, 0.580),  
                (0.57, 0.520), (0.595, 0.500), (0.615, 0.500), (0.66, 0.550),  
                (0.73, 0.640), (0.81, 0.760), (0.88, 0.880), (0.95, 1.000)]],  
        cut=[]),
```

```
    # Haze and flat overcast light, very low edge contrast.
```

```
    "lowlight_01.jpg": dict(  
        road=[[(0.03, 1.000), (0.18, 0.900), (0.30, 0.820), (0.38, 0.770),  
                (0.435, 0.735), (0.465, 0.735), (0.55, 0.780), (0.68, 0.850),  
                (0.82, 0.920), (0.95, 1.000)]],  
        cut=[]),
```

```
    # Overcast, weak markings, road runs away up and slightly to the left.
```

```
    "lowlight_02.jpg": dict(  
        road=[[(0.170, 1.000), (0.235, 0.900), (0.300, 0.800), (0.350, 0.700),  
                (0.395, 0.600), (0.425, 0.520), (0.433, 0.490), (0.438, 0.490),  
                (0.450, 0.600), (0.485, 0.700), (0.525, 0.800), (0.575, 0.900),  
                (0.620, 1.000)]],  
        cut=[[(0.415, 0.870), (0.505, 0.870), (0.505, 1.000), (0.415, 1.000)]],
```

```
    # Strong single bend with clear markings.
```

```
    "curve_01.jpg": dict(  
        road=[[(0.00, 0.660), (0.12, 0.645), (0.25, 0.632), (0.36, 0.620),  
                (0.45, 0.612), (0.52, 0.620), (0.58, 0.645), (0.65, 0.685),  
                (0.72, 0.745), (0.79, 0.830), (0.85, 0.920), (0.88, 1.000),  
                (0.00, 1.000)]],  
        cut=[]),
```

```
    # Sweeping curve seen from the inside of the bend.
```

```
    "curve_02.jpg": dict(  
        road=[[(0.42, 1.000), (0.50, 0.900), (0.555, 0.800), (0.585, 0.720),  
                (0.605, 0.655), (0.635, 0.595), (0.70, 0.585), (0.78, 0.605),  
                (0.88, 0.635), (1.00, 0.665), (1.00, 1.000)]],  
        cut=[]),
```

```
    # Unsealed laterite road, no markings at all; a car is parked far ahead.
```

```
    "worn_01.jpg": dict(  
        road=[[(0.10, 1.000), (0.22, 0.880), (0.33, 0.770), (0.43, 0.680),  
                (0.49, 0.630), (0.52, 0.605), (0.545, 0.605), (0.60, 0.640),  
                (0.68, 0.710), (0.78, 0.800), (0.88, 0.900), (0.97, 1.000)]],  
        cut=[[(0.505, 0.560), (0.585, 0.560), (0.585, 0.615), (0.505, 0.615)]],
```

```
    # Straight laterite road; the boundary is a colour change, not an edge line.
```

```
    "worn_02.jpg": dict(  
        road=[[(0.280, 1.000), (0.330, 0.900), (0.390, 0.800), (0.450, 0.700),  
                (0.495, 0.645), (0.515, 0.620), (0.535, 0.620), (0.565, 0.645),  
                (0.620, 0.700), (0.700, 0.800), (0.760, 0.900), (0.800, 1.000)]],  
        cut=[]),
```

```
    # A queue of minibuses standing on the carriageway.
```

```

"occlusion_01.jpg": dict(
    road=[[ (0.00, 0.600), (0.12, 0.645), (0.24, 0.680), (0.34, 0.720),
            (0.44, 0.775), (0.52, 0.820), (0.60, 0.870), (0.68, 0.920),
            (0.75, 0.955), (0.80, 0.900), (0.85, 0.800), (0.88, 0.720),
            (0.92, 0.700), (1.00, 0.700), (1.00, 1.000), (0.00, 1.000)]],
    cut=[]),

# A tractor and a pickup occupying the carriageway ahead.
"occlusion_02.jpg": dict(
    road=[[ (0.13, 1.000), (0.15, 0.850), (0.17, 0.740), (0.22, 0.660),
            (0.30, 0.620), (0.42, 0.600), (0.55, 0.600), (0.62, 0.620),
            (0.70, 0.640), (0.80, 0.660), (0.90, 0.680), (1.00, 0.700),
            (1.00, 1.000)]],
    cut=[[ (0.330, 0.590), (0.595, 0.590), (0.595, 0.760), (0.330, 0.760)]]),
}

# -----
def to_px(poly, w, h):
    return np.array([[int(round(x * w)), int(round(y * h))] for x, y in poly], np.int32)

def build_mask(shape, spec):
    h, w = shape[:2]
    mask = np.zeros((h, w), np.uint8)
    for poly in spec["road"]:
        cv2.fillPoly(mask, [to_px(poly, w, h)], 255)
    for poly in spec.get("cut", []):
        cv2.fillPoly(mask, [to_px(poly, w, h)], 0)
    return mask

def overlay(img, mask, colour=(0, 200, 0), alpha=0.40):
    out = img.copy()
    out[mask > 0] = (np.array(colour) * alpha
                    + out[mask > 0] * (1 - alpha)).astype(np.uint8)
    cnts, _ = cv2.findContours((mask > 0).astype(np.uint8), cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
    cv2.drawContours(out, cnts, -1, (0, 0, 255), 2)
    return out

def main():
    os.makedirs(ANN_DIR, exist_ok=True)
    os.makedirs(CHECK_DIR, exist_ok=True)
    summary = {}
    for name, spec in ANNOTATION.items():
        img = cv2.imread(os.path.join(IMG_DIR, name), cv2.IMREAD_COLOR)
        if img is None:
            raise IOError("cannot read " + name)
        mask = build_mask(img.shape, spec)
        cv2.imwrite(os.path.join(ANN_DIR, name.replace(".jpg", "_road.png")), mask)
        cv2.imwrite(os.path.join(CHECK_DIR, "gt_" + name),
                    overlay(img, mask), [cv2.IMWRITE_JPEG_QUALITY, 92])
        frac = float((mask > 0).mean())
        summary[name] = {"road_fraction": round(frac, 4),
                        "pixels": int((mask > 0).sum()),
                        "size": [int(img.shape[1]), int(img.shape[0])],
                        "polygons": len(spec["road"]), "cuts": len(spec.get("cut", []))}
    print("%-18s road covers %5.1f%% of the frame" % (name, 100 * frac))
    with open(os.path.join(ANN_DIR, "annotation_summary.json"), "w",
              encoding="utf-8") as fh:
        json.dump(summary, fh, indent=2)

if __name__ == "__main__":
    main()

```

src/preprocessing.py

```

"""
preprocessing.py
Colour representation and noise suppression.

```

Two questions are settled here, and both are settled by measurement rather than by assertion:

- which colour representation separates road from not road, and
- which filter suppresses noise without destroying the road boundary.

Colour. Road surfaces in these photographs are grey asphalt or orange laterite. What they have in common is not a colour but a *lack* of one: compared with the vegetation, sky and buildings around them they are weakly saturated and their lightness is mid range. HLS exposes exactly those two quantities on separate axes, and the L channel of HLS is also where white and yellow lane paint stands out most strongly. Lab is kept as the alternative because its a and b axes are approximately perceptually uniform, which matters for clustering. RGB is kept only as the baseline that mixes brightness and chroma together and therefore moves under every change of illumination.

Filtering. A road boundary is a step edge that the edge detector must keep, while the texture of chippings, gravel and tree shade is noise that it must lose. Gaussian smoothing treats both the same way. A bilateral filter weights by intensity difference as well as distance, so it smooths within a region and stops at the boundary between regions. The two are compared directly in experiment E2.

```

from __future__ import annotations

```

```

import cv2
import numpy as np

# Names used consistently in the tables and the report.
COLOUR_SPACES = ["BGR", "HLS", "Lab", "HSV"]
FILTERS = ["none", "gaussian", "median", "bilateral"]

def to_space(bgr, space):
    """Convert to one of the colour representations under test."""
    if space == "BGR":
        return bgr.copy()
    if space == "HLS":
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2HLS)
    if space == "Lab":
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2Lab)
    if space == "HSV":
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    raise ValueError("unknown colour space " + space)

def apply_filter(bgr, kind, strength=1.0):
    """Suppress noise. `strength` scales the kernel so it can be swept.

    Gaussian : separable, isotropic, cheap, blurs edges along with noise.
    Median : removes impulsive speckle, preserves step edges, slower.
    Bilateral : edge preserving, the most expensive of the three.
    """
    if kind == "none":
        return bgr.copy()
    if kind == "gaussian":
        k = int(max(1, round(5 * strength)) * 2 + 1)
        return cv2.GaussianBlur(bgr, (k, k), 0)
    if kind == "median":
        k = int(max(1, round(2 * strength)) * 2 + 1)
        return cv2.medianBlur(bgr, k)
    if kind == "bilateral":
        d = int(max(3, round(9 * strength)))
        return cv2.bilateralFilter(bgr, d, 75 * strength, 75 * strength)
    raise ValueError("unknown filter " + kind)

def road_channel(bgr, space="HLS"):
    """Return the single channel in which the road is most distinctive.

    For HLS that is lightness, which carries the road surface and the paint.
    For Lab it is L. For HSV it is V. For BGR the grey scale conversion is
    used, which is the honest baseline.
    """
    if space == "BGR":
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    conv = to_space(bgr, space)
    if space == "HLS":
        return conv[:, :, 1]
    if space == "Lab":
        return conv[:, :, 0]
    if space == "HSV":
        return conv[:, :, 2]
    raise ValueError(space)

def separability(bgr, mask, space):
    """How well does this colour representation separate road from not road?

    Reports the Fisher ratio between the road and non road populations,

    
$$J = (\mu_{road} - \mu_{other})^2 / (\text{var}_{road} + \text{var}_{other})$$


    averaged over the channels of the representation. A larger value means the
    two populations are further apart relative to their spread, so a threshold
    or a clustering step has an easier job. This is computed with the manual
    ground truth and is therefore a property of the *representation*, not of any
    particular segmentation method.
    """
    conv = to_space(bgr, space).astype(np.float32)
    road = mask > 0
    other = ~road
    if road.sum() < 50 or other.sum() < 50:
        return 0.0
    scores = []
    for c in range(conv.shape[2]):
        ch = conv[:, :, c]
        a, b = ch[road], ch[other]
        denom = a.var() + b.var()
        if denom < 1e-6:
            continue
        scores.append((a.mean() - b.mean()) ** 2 / denom)
    return float(np.mean(scores)) if scores else 0.0

def illumination_correction(bgr, clip=2.0, tiles=8):
    """Local contrast equalisation on the lightness channel only.

    Applied to the hazy and heavily shaded photographs. Working on L in HLS
    rather than on the three BGR channels avoids shifting the hue, which would
    defeat the colour based segmentation that follows.
    """
    hls = cv2.cvtColor(bgr, cv2.COLOR_BGR2HLS)
    clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tiles, tiles))
    hls[:, :, 1] = clahe.apply(hls[:, :, 1])

```

```
return cv2.cvtColor(hls, cv2.COLOR_HLS2BGR)
```

src/edges.py

```
"""
edges.py
Edge detection and the region of interest.

Edge detection. Two operators are implemented and compared. Sobel estimates
the image gradient with a small separable kernel and thresholds its magnitude,
which is the simplest thing that can work and returns thick, noisy responses.
Canny adds the three steps that make an edge detector usable: smoothing,
non maximum suppression across the gradient direction so that edges come out
one pixel wide, and hysteresis thresholding so that a weak edge is kept when it
connects to a strong one. Hysteresis is the reason Canny survives a faded lane
marking that a single threshold on Sobel magnitude either loses entirely or
buries in noise.

Region of interest. Almost everything above the horizon is irrelevant, and it
is also where most of the strong edges live: trees, buildings, poles and the
skyline. Restricting the search to a trapezoid that covers the carriageway in
front of the camera removes those distractors before they can be fitted, at the
cost of one strong assumption, that the camera points along the road. Section 6
of the report shows what that assumption costs on the curved scenes.
"""

from __future__ import annotations

import cv2
import numpy as np

EDGE_METHODS = ["canny", "sobel"]

# Trapezoid corners as fractions of the frame, (x, y), y downwards.
# Wide at the bottom of the frame, narrow near the horizon.
DEFAULT_ROI = dict(bottom_left=0.00, bottom_right=1.00,
                   top_left=0.28, top_right=0.72, top=0.45)

def roi_polygon(shape, roi=None):
    """Return the region of interest trapezoid in pixels."""
    roi = dict(DEFAULT_ROI, **(roi or {}))
    h, w = shape[:2]
    return np.array([[
        (int(roi["bottom_left"] * w), h - 1),
        (int(roi["top_left"] * w), int(roi["top"] * h)),
        (int(roi["top_right"] * w), int(roi["top"] * h)),
        (int(roi["bottom_right"] * w), h - 1),
    ]], np.int32)

def roi_mask(shape, roi=None):
    m = np.zeros(shape[:2], np.uint8)
    cv2.fillPoly(m, roi_polygon(shape, roi), 255)
    return m

def apply_roi(img, roi=None):
    return cv2.bitwise_and(img, img, mask=roi_mask(img.shape, roi))

def detect_edges(grey, method="canny", low=50, high=150, sobel_k=3,
                 sobel_thresh=60):
    """Return a binary edge map.

    canny : low and high are the hysteresis thresholds.
    sobel : the gradient magnitude is thresholded at sobel_thresh, with no
            thinning and no hysteresis, which is exactly the point of the
            comparison.
    """
    if method == "canny":
        return cv2.Canny(grey, low, high, apertureSize=3, L2gradient=True)
    if method == "sobel":
        gx = cv2.Sobel(grey, cv2.CV_32F, 1, 0, ksize=sobel_k)
        gy = cv2.Sobel(grey, cv2.CV_32F, 0, 1, ksize=sobel_k)
        mag = cv2.magnitude(gx, gy)
        mag = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
        return (mag >= sobel_thresh).astype(np.uint8) * 255
    raise ValueError("unknown edge method " + method)

def edge_density(edge_map, mask=None):
    """Fraction of pixels that the operator declared to be edges."""
    if mask is not None:
        sel = mask > 0
        if sel.sum() == 0:
            return 0.0
        return float((edge_map[sel] > 0).mean())
    return float((edge_map > 0).mean())

def boundary_precision(edge_map, gt_mask, tolerance=4, mask=None):
    """Of the edges the operator reported, how many lie on the true boundary?

    The complement of boundary_recall. Without it a detector can score well by
    simply declaring everything to be an edge, which is the degenerate case the
    Sobel operator falls into at a low threshold.
    """
    gt_edge = cv2.Canny((gt_mask > 0).astype(np.uint8) * 255, 50, 150)
    k = np.ones((2 * tolerance + 1, 2 * tolerance + 1), np.uint8)
```

```

near_gt = cv2.dilate(gt_edge, k) &gt; 0
det = edge_map &gt; 0
if mask is not None:
    det = det &amp; (mask &gt; 0)
total = det.sum()
if total == 0:
    return 0.0
return float((det &amp; near_gt).sum() / total)

def boundary_f1(edge_map, gt_mask, tolerance=4, mask=None):
    """Harmonic mean of boundary recall and boundary precision."""
    r = boundary_recall(edge_map, gt_mask, tolerance)
    p = boundary_precision(edge_map, gt_mask, tolerance, mask)
    return 0.0 if (p + r) == 0 else 2 * p * r / (p + r)

def boundary_recall(edge_map, gt_mask, tolerance=4):
    """How much of the true road boundary did the operator actually find?

    The ground truth boundary is dilated by `tolerance` pixels and the
    fraction of it that coincides with a detected edge is reported. This
    measures the edge stage on its own terms, before any line fitting, and is
    what separates a detector that is merely busy from one that is useful.
    """
    gt_edge = cv2.Canny((gt_mask &gt; 0).astype(np.uint8) * 255, 50, 150)
    k = np.ones((2 * tolerance + 1, 2 * tolerance + 1), np.uint8)
    found = cv2.dilate(edge_map, k) &gt; 0
    total = (gt_edge &gt; 0).sum()
    if total == 0:
        return 0.0
    return float((found &amp; (gt_edge &gt; 0)).sum() / total)

```

src/lanes.py

```

"""
lanes.py
Identifying candidate lane and road boundaries from the edge map.

The probabilistic Hough transform turns a cloud of edge pixels into line
segments. Each segment votes for a line in parameter space, and a line is
reported when enough collinear pixels agree, which is what makes the method
tolerant of the gaps in a dashed centre line and of the pixels an occluding
vehicle removes.

Segments are then sorted into a left and a right boundary by the sign of their
slope, because in a forward facing view the left boundary recedes up and to the
right and the right boundary recedes up and to the left. Nearly horizontal
segments are discarded: they belong to shadows lying across the road, to the
horizon, and to the backs of vehicles, none of which is a lane boundary. Each
side is then reduced to a single straight line by a least squares fit weighted
by segment length, so that long confident segments outvote short ones.

The straight line model is the weak point of the whole stage and the report
does not hide it: on a curve the true boundary is not a line, and Section 6
measures exactly how much accuracy that costs.
"""

from __future__ import annotations

import cv2
import numpy as np

MIN_ABS_SLOPE = 0.35 # below this a segment is treated as horizontal
DEFAULT_HOUGH = dict(rho=1, theta=np.pi / 180, threshold=40,
                     min_line_len=40, max_line_gap=120)

def hough_segments(edge_map, params=None):
    """Return the raw probabilistic Hough segments as an (N, 4) array."""
    p = dict(DEFAULT_HOUGH, **(params or {}))
    lines = cv2.HoughLinesP(edge_map, p["rho"], p["theta"], p["threshold"],
                           minLineLength=p["min_line_len"],
                           maxLineGap=p["max_line_gap"])

    if lines is None:
        return np.zeros((0, 4), np.int32)
    return lines.reshape(-1, 4)

def split_by_slope(segments, width):
    """Split segments into left boundary, right boundary and rejected."""
    left, right, rejected = [], [], []
    for x1, y1, x2, y2 in segments:
        if x2 == x1:
            rejected.append((x1, y1, x2, y2))
            continue
        slope = (y2 - y1) / float(x2 - x1)
        if abs(slope) < MIN_ABS_SLOPE:
            rejected.append((x1, y1, x2, y2))
        elif slope < 0 and min(x1, x2) < 0.95 * width:
            left.append((x1, y1, x2, y2))
        elif slope > 0 and max(x1, x2) > 0.05 * width:
            right.append((x1, y1, x2, y2))
        else:
            rejected.append((x1, y1, x2, y2))
    return left, right, rejected

def fit_boundary(segments, shape, y_top_frac=0.58):
    """Fit one straight boundary to a set of segments.

```

```

Each segment contributes its two endpoints, weighted by its length, and a
line  $x = m \cdot y + c$  is fitted by weighted least squares. Fitting  $x$  as a
function of  $y$  rather than the other way round keeps the fit stable for the
near vertical boundaries that matter here.
"""
# One stray segment is not evidence of a boundary. Requiring at least two
# stops the fit from extrapolating a whole boundary out of a single edge on
# a roadside bush.
if len(segments) < 2:
    return None
h, w = shape[0], shape[1]
xs, ys, ws = [], [], []
for x1, y1, x2, y2 in segments:
    length = float(np.hypot(x2 - x1, y2 - y1))
    xs += [x1, x2]
    ys += [y1, y2]
    ws += [length, length]
xs, ys, ws = np.array(xs, float), np.array(ys, float), np.array(ws, float)
if len(xs) < 2 or ys.max() - ys.min() < 5:
    return None
A = np.stack([ys, np.ones_like(ys)], axis=1)
W = np.diag(ws)
try:
    coef = np.linalg.lstsq(W @ A, W @ xs, rcond=None)[0]
except np.linalg.LinAlgError:
    return None
m, c = coef
y_bot, y_top = h - 1, int(y_top_frac * h)
x_bot, x_top = m * y_bot + c, m * y_top + c
# A boundary that leaves the frame by more than half its width is not a
# boundary, it is an artefact of extrapolating a short segment. Reporting
# nothing is more useful than reporting that.
if not (-0.5 * w <= x_bot <= 1.5 * w and -0.5 * w <= x_top <= 1.5 * w):
    return None
return (int(x_bot), y_bot, int(x_top), y_top)

def detect_lanes(edge_map, shape, params=None, y_top_frac=0.58, restrict=None):
    """Full boundary identification stage.

    `restrict` optionally gates the edge map, for example with the dilated
    paint mask, so that only edges lying on road markings can vote.
    """
    if restrict is not None:
        edge_map = cv2.bitwise_and(edge_map, restrict)
    segs = hough_segments(edge_map, params)
    left, right, rejected = split_by_slope(segs, shape[1])
    return {
        "segments": segs,
        "left_segments": left,
        "right_segments": right,
        "rejected": rejected,
        "left": fit_boundary(left, shape, y_top_frac),
        "right": fit_boundary(right, shape, y_top_frac),
    }

def boundary_error(fitted, gt_mask, side, rows=12):
    """Mean horizontal error of a fitted boundary, in pixels.

    The ground truth boundary at a given row is the leftmost or rightmost road
    pixel in that row of the manual mask. The fitted line is evaluated at the
    same rows and the mean absolute difference is reported. Rows where the mask
    has no road are skipped. Returns None when nothing could be compared.
    """
    if fitted is None:
        return None
    h, w = gt_mask.shape[:2]
    x1, y1, x2, y2 = fitted
    if y1 == y2:
        return None
    m = (x2 - x1) / float(y2 - y1)
    errs = []
    for yf in np.linspace(0.55, 0.95, rows):
        y = int(yf * (h - 1))
        cols = np.where(gt_mask[y] > 0)[0]
        if len(cols) < 5:
            continue
        # A road that runs out of the frame has no visible boundary on that
        # side in that row, so there is nothing to compare the fit against.
        if side == "left" and cols.min() <= 2:
            continue
        if side == "right" and cols.max() >= w - 3:
            continue
        gt_x = cols.min() if side == "left" else cols.max()
        errs.append(abs(x1 + m * (y - y1) - gt_x))
    if len(errs) < 3:
        return None
    return float(np.mean(errs))

def draw_lanes(img, result, colour=(0, 0, 255), thickness=6):
    out = img.copy()
    for key in ("left", "right"):
        line = result.get(key)
        if line is not None:
            cv2.line(out, (line[0], line[1]), (line[2], line[3]), colour, thickness)
    return out

def draw_segments(img, result):
    """Colour coded Hough segments: kept left, kept right and rejected."""

```

```

out = img.copy()
for x1, y1, x2, y2 in result["rejected"]:
    cv2.line(out, (x1, y1), (x2, y2), (120, 120, 120), 2)
for x1, y1, x2, y2 in result["left_segments"]:
    cv2.line(out, (x1, y1), (x2, y2), (0, 200, 255), 3)
for x1, y1, x2, y2 in result["right_segments"]:
    cv2.line(out, (x1, y1), (x2, y2), (255, 120, 0), 3)
return out

# -----
# Lane marking isolation
# -----
def paint_mask(bgr, white_l=170, white_s=90, yellow_lo=(15, 70, 70),
               yellow_hi=(38, 255, 255), dilate=9):
    """Isolate white and yellow road paint in HLS.
    Raw edges inside the region of interest are dominated by things that are
    not lane boundaries: the texture of the surface, the shadows of roadside
    trees, vegetation and the backs of vehicles. Road paint, however, is one of
    only two colours, and both are easy to state in HLS. White paint is high
    lightness and low saturation; yellow paint occupies a narrow hue band with
    a decent saturation.

    The result is dilated so that it can be used as a permissive gate on the
    edge map: an edge is kept when it lies on or beside paint. Roads with no
    paint at all return a nearly empty mask, which is the honest answer and is
    why the unsealed scenes fall back on the region derived boundary below.
    """
    hls = cv2.cvtColor(bgr, cv2.COLOR_BGR2HLS)
    white = ((hls[:, :, 1] >= white_l) & (hls[:, :, 2] <= white_s))
    yellow = cv2.inRange(hls, np.array(yellow_lo, np.uint8),
                          np.array(yellow_hi, np.uint8)) & 0
    m = ((white | yellow).astype(np.uint8)) * 255
    if dilate:
        k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (dilate, dilate))
        m = cv2.dilate(m, k, iterations=1)
    return m

def boundaries_from_region(road_mask, y_top_frac=0.58, rows=24):
    """Derive the left and right boundaries from the segmented road region.

    For each sampled row the leftmost and rightmost road pixels are taken as
    boundary observations and a straight line is fitted to each side by the
    same weighted least squares used for the Hough segments. Rows where the
    region runs off the side of the frame are skipped, because there the
    boundary is the edge of the photograph and not the edge of the road.

    This path needs no paint and no edges at all, so it is the one that still
    works on the unsealed roads. What it cannot do is find a lane *within* a
    road: it only ever reports the outside of the drivable region.
    """
    h, w = road_mask.shape[:2]
    left_pts, right_pts = [], []
    for yf in np.linspace(y_top_frac, 0.99, rows):
        y = int(yf * (h - 1))
        cols = np.where(road_mask[y] > 0)[0]
        if len(cols) < 10:
            continue
        if cols.min() > 2:
            left_pts.append((int(cols.min()), y))
        if cols.max() < w - 3:
            right_pts.append((int(cols.max()), y))

    def fit(points):
        if len(points) < 4:
            return None
        xs = np.array([p[0] for p in points], float)
        ys = np.array([p[1] for p in points], float)
        if ys.max() - ys.min() < 5:
            return None
        A = np.stack([ys, np.ones_like(ys)], axis=1)
        m, c = np.linalg.lstsq(A, xs, rcond=None)[0]
        y_bot, y_top = h - 1, int(y_top_frac * h)
        return (int(m * y_bot + c), y_bot, int(m * y_top + c), y_top)

    return {"left": fit(left_pts), "right": fit(right_pts),
            "left_points": left_pts, "right_points": right_pts,
            "segments": np.zeros((0, 4), np.int32),
            "left_segments": [], "right_segments": [], "rejected": []}

```

src/segmentation.py

```

"""
segmentation.py
Separating the drivable road region from the rest of the scene.

```

Three classical approaches are implemented and compared. All three are given the same input and the same post processing, so the comparison isolates the segmentation rule itself.

1. Adaptive colour thresholding. A sample of pixels is taken from a small trapezoid immediately in front of the camera, which is the one place a forward facing photograph is almost certain to be road. The mean and spread of that sample in HLS define a per image acceptance band, and every pixel within it is called road. This is thresholding, but with the threshold set from the image rather than fixed in advance, which is what lets one rule survive both grey asphalt and orange laterite.
2. K means clustering in Lab. The image is clustered into K colours and the

cluster that dominates the same near field sample is taken to be the road. No threshold is chosen at all: the data decides where the boundaries lie. The cost is that K must be chosen, and that a cluster can straddle road and shoulder when the two are similar.

3. Mahalanobis thresholding in Lab. The same near field sample is used, but instead of an independent band per channel the full 3x3 covariance of the sample is estimated and a pixel is accepted when its Mahalanobis distance from the sample mean is below a threshold. Treating the channels independently, as approach 1 does, implicitly assumes the acceptance region is an axis aligned box; in practice the road colours form a tilted, elongated cloud, and a box that covers it must also admit a great deal that is not road. This is the same idea as approach 1 with the correlation put back in, and experiment E6 shows it is worth a large amount of accuracy.
4. Region growing. A flood fill grows outwards from seeds in the near field, accepting a neighbour when it is within a tolerance of the seed region. Unlike the other two this enforces spatial connectivity, so a patch of grey roof on the far side of a hedge cannot be labelled road. That is its main advantage and, under a shadow that cuts the road in half, its main weakness.

All three are followed by the same morphological cleanup and by keeping the connected component that touches the bottom of the frame, since the drivable region must by definition reach the vehicle.

```

"""
from __future__ import annotations

import cv2
import numpy as np

METHODS = ["threshold", "mahalanobis", "kmeans", "region_growing"]

def near_field_sample(shape, width=0.22, height=0.10):
    """A trapezoid at the bottom centre, used as the road colour prior."""
    h, w = shape[:2]
    cx = w // 2
    half = int(width * w / 2)
    y0 = int((1.0 - height) * h)
    return np.array([[(cx - half, h - 1), (cx - int(half * 0.7), y0),
                     (cx + int(half * 0.7), y0), (cx + half, h - 1)], np.int32])

def _sample_mask(shape, width=0.22, height=0.10):
    m = np.zeros(shape[:2], np.uint8)
    cv2.fillPoly(m, near_field_sample(shape, width, height), 255)
    return m

def horizon_cut(mask, frac=0.35):
    """Discard everything above the horizon.

    This is the region selection step applied to segmentation. Sky, distant
    buildings and tree canopy are never drivable, and a surprising amount of
    sky is close enough to grey asphalt in colour to be accepted by any of the
    rules below. Cutting the top of the frame removes that failure mode at the
    cost of one assumption: the camera is roughly level.
    """
    out = mask.copy()
    out[:int(frac * mask.shape[0])] = 0
    return out

def segment_mahalanobis(bgr, chi=3.0, cleanup=11, horizon=0.35, ridge=4.0):
    """Multivariate colour thresholding in Lab.

    The near field sample gives a mean and a full covariance, and a pixel is
    called road when

    
$$\sqrt{(x - \mu)^T C^{-1} (x - \mu)} \leq \chi$$


    which accepts an ellipsoid oriented along the correlations actually present
    in the road colours rather than an axis aligned box. A small ridge is added
    to the diagonal of C so that a very uniform sample, which happens on a
    smooth asphalt surface, cannot produce a near singular covariance and an
    absurdly narrow acceptance region.
    """
    lab = cv2.cvtColor(cv2.GaussianBlur(bgr, (7, 7), 0),
                       cv2.COLOR_BGR2Lab).astype(np.float32)
    h, w = bgr.shape[:2]
    sample = _sample_mask(bgr.shape) &gt; 0
    if sample.sum() &lt; 50:
        return np.zeros((h, w), np.uint8)
    X = lab[sample]
    mu = X.mean(axis=0)
    C = np.cov(X.T) + np.eye(3) * ridge
    try:
        Ci = np.linalg.inv(C)
    except np.linalg.LinAlgError:
        return np.zeros((h, w), np.uint8)
    d = lab.reshape(-1, 3) - mu
    md = np.sqrt(np.einsum("ij,jk,ik->i", d, Ci, d)).reshape(h, w)
    mask = (md &lt;= chi).astype(np.uint8) * 255
    if horizon:
        mask = horizon_cut(mask, horizon)
    return _cleanup(mask, kernel=cleanup)

def _cleanup(mask, keep_bottom=True, kernel=11):

```

```

k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel, kernel))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k, iterations=1)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k, iterations=2)
if not keep_bottom:
    return mask
n, lab, stats, _ = cv2.connectedComponentsWithStats((mask &gt; 0).astype(np.uint8), 8)
if n &lt;= 1:
    return mask
h = mask.shape[0]
bottom = lab[h - 1, :]
ids = [i for i in np.unique(bottom) if i != 0]
if not ids:
    # nothing reaches the camera
    idx = 1 + int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
    ids = [idx]
out = np.zeros_like(mask)
for i in ids:
    out[lab == i] = 255
return out

def segment_threshold(bgr, k_sigma=2.5, cleanup=11):
    """Adaptive colour band in HLS, set from the near field sample."""
    hls = cv2.cvtColor(bgr, cv2.COLOR_BGR2HLS).astype(np.float32)
    sample = _sample_mask(bgr.shape) &gt; 0
    if sample.sum() &lt; 50:
        return np.zeros(bgr.shape[:2], np.uint8)
    mask = np.ones(bgr.shape[:2], bool)
    for c, spread_floor in ((0, 6.0), (1, 12.0), (2, 10.0)):
        ch = hls[:, :, c]
        vals = ch[sample]
        mu, sd = float(vals.mean()), max(float(vals.std()), spread_floor)
        mask &amp;= np.abs(ch - mu) &lt;= k_sigma * sd
    return _cleanup((mask.astype(np.uint8) * 255), kernel=cleanup)

def segment_kmeans(bgr, k=3, attempts=3, cleanup=11, downscale=0.5):
    """Cluster in Lab and keep the cluster that owns the near field."""
    small = cv2.resize(bgr, None, fx=downscale, fy=downscale,
        interpolation=cv2.INTER_AREA)
    lab = cv2.cvtColor(small, cv2.COLOR_BGR2Lab).reshape(-1, 3).astype(np.float32)
    crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)
    _, labels, _ = cv2.kmeans(lab, k, None, crit, attempts,
        cv2.KMEANS_PP_CENTERS)
    labels = labels.reshape(small.shape[:2])
    labels = cv2.resize(labels.astype(np.int32), (bgr.shape[1], bgr.shape[0]),
        interpolation=cv2.INTER_NEAREST)
    sample = _sample_mask(bgr.shape) &gt; 0
    if sample.sum() &lt; 50:
        return np.zeros(bgr.shape[:2], np.uint8)
    vals, counts = np.unique(labels[sample], return_counts=True)
    road_label = int(vals[int(np.argmax(counts))])
    return _cleanup(((labels == road_label).astype(np.uint8) * 255), kernel=cleanup)

def segment_region_growing(bgr, tolerance=14, cleanup=11, seeds=5):
    """Flood fill outwards from seeds on the near field road surface."""
    blur = cv2.GaussianBlur(bgr, (7, 7), 0)
    h, w = bgr.shape[:2]
    filled = blur.copy()
    ff_mask = np.zeros((h + 2, w + 2), np.uint8)
    lo = (tolerance,) * 3
    hi = (tolerance,) * 3
    for i in range(seeds):
        fx = 0.5 + (i - (seeds - 1) / 2.0) * 0.04
        px = int(np.clip(fx, 0.02, 0.98) * w)
        py = int(0.96 * h)
        cv2.floodFill(filled, ff_mask, (px, py), (255, 255, 255), lo, hi,
            cv2.FLOODFILL_FIXED_RANGE | 8 | (255 &lt;&lt; 8))
    grown = (ff_mask[1:-1, 1:-1] &gt; 0).astype(np.uint8) * 255
    return _cleanup(grown, kernel=cleanup)

def segment(bgr, method="threshold", **kw):
    if method == "threshold":
        return segment_threshold(bgr, **kw)
    if method == "mahalanobis":
        return segment_mahalanobis(bgr, **kw)
    if method == "kmeans":
        return segment_kmeans(bgr, **kw)
    if method == "region_growing":
        return segment_region_growing(bgr, **kw)
    raise ValueError("unknown segmentation method " + method)

def restrict_to_roi(mask, roi_mask_img):
    """Optionally confine a segmentation to the region of interest."""
    return cv2.bitwise_and(mask, roi_mask_img)

def overlay_result(bgr, road_mask, lanes=None, colour=(0, 200, 0), alpha=0.40):
    """Final visualisation: road region tinted, boundaries drawn over it."""
    out = bgr.copy()
    sel = road_mask &gt; 0
    out[sel] = (np.array(colour) * alpha + out[sel] * (1 - alpha)).astype(np.uint8)
    cnts, _ = cv2.findContours((road_mask &gt; 0).astype(np.uint8),
        cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cv2.drawContours(out, cnts, -1, (0, 140, 0), 2)
    if lanes:
        for key in ("left", "right"):
            line = lanes.get(key)

```

```

        if line is not None:
            cv2.line(out, (line[0], line[1]), (line[2], line[3]), (0, 0, 255), 6)
    return out

```

src/evaluate.py

```

"""
evaluate.py
Quantitative evaluation against the manual ground truth.

Segmentation is scored per pixel against the hand traced masks:

    IoU      = TP / (TP + FP + FN)
    precision = TP / (TP + FP)      of the pixels called road, how many were
    recall    = TP / (TP + FN)      of the road pixels, how many were found
    F1        = 2PR / (P + R)

IoU is reported first because it is the measure that punishes both kinds of
mistake at once and cannot be inflated by predicting a very large or a very
small region, which precision and recall can be individually.

Boundary accuracy is scored separately in lanes.boundary_error, in pixels,
because a detector can enclose the right area while placing its edges badly.
"""

from __future__ import annotations

import csv
import json
import os

import numpy as np

def confusion(pred, gt):
    p = pred &gt; 0
    g = gt &gt; 0
    tp = int(np.logical_and(p, g).sum())
    fp = int(np.logical_and(p, ~g).sum())
    fn = int(np.logical_and(~p, g).sum())
    tn = int(np.logical_and(~p, ~g).sum())
    return tp, fp, fn, tn

def scores(pred, gt):
    tp, fp, fn, tn = confusion(pred, gt)
    iou = tp / float(tp + fp + fn) if (tp + fp + fn) else 0.0
    precision = tp / float(tp + fp) if (tp + fp) else 0.0
    recall = tp / float(tp + fn) if (tp + fn) else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if (precision + recall) else 0.0)
    return {"iou": iou, "precision": precision, "recall": recall, "f1": f1,
            "tp": tp, "fp": fp, "fn": fn, "tn": tn}

def mean_scores(rows, keys=("iou", "precision", "recall", "f1")):
    out = {}
    for k in keys:
        vals = [r[k] for r in rows if r.get(k) is not None]
        out["mean_" + k] = float(np.mean(vals)) if vals else 0.0
    return out

def write_csv(path, header, rows):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(header)
        for r in rows:
            w.writerow(["%.4f" % v if isinstance(v, float) else v for v in r])

def write_json(path, obj):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8") as fh:
        json.dump(obj, fh, indent=2, default=str)

```

src/run_experiments.py

```

"""
run_experiments.py
Runs every experiment and writes the tables, figures and overlays.

python src/run_experiments.py

Protocol. The twelve photographs are split in half. The six images whose name
ends in _01 are the tuning set: every parameter reported below was chosen on
those and only those. The six ending in _02 are held out and were run once,
at the end, with the parameters already fixed. Both halves cover all six
imaging conditions, so the held out half is not easier than the tuning half.

Experiments
E1 colour representation, measured by class separability
E2 filtering: four filters against boundary survival and noise
E3 edge detection: Canny against Sobel, and their threshold sweeps
E4 the region of interest: what it removes and what it costs

```

```

E5 lane boundary fitting and the Hough parameters
E6 segmentation: three approaches and their parameters
E7 the assembled pipeline, on the tuning half and the held out half
"""

from __future__ import annotations

import json
import os
import time

import cv2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

import edges as E
import evaluate as EV
import lanes as L
import preprocessing as P
import segmentation as S

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
IMG_DIR = os.path.join(ROOT, "data", "images")
ANN_DIR = os.path.join(ROOT, "data", "annotations")
RES = os.path.join(ROOT, "results")
FIG = os.path.join(RES, "figures")
TAB = os.path.join(RES, "tables")
OVL = os.path.join(RES, "overlays")
for d in (RES, FIG, TAB, OVL):
    os.makedirs(d, exist_ok=True)

plt.rcParams.update({"figure.dpi": 130, "font.size": 9, "axes.grid": True,
                    "grid.alpha": 0.3, "savefig.bbox": "tight"})

CONDITION = {
    "daylight": "good daylight", "shadows": "shadows",
    "poor_illumination": "poor illumination", "curve": "curved road",
    "worn_markings": "worn or absent markings", "occlusion": "occluding traffic",
}
SUMMARY = {}

# -----
def load_dataset():
    with open(os.path.join(ROOT, "data", "sources.json"), encoding="utf-8") as fh:
        meta = {r["file"]: r for r in json.load(fh)}
    items = []
    for name in sorted(os.listdir(IMG_DIR)):
        if not name.endswith(".jpg"):
            continue
        img = cv2.imread(os.path.join(IMG_DIR, name), cv2.IMREAD_COLOR)
        gt = cv2.imread(os.path.join(ANN_DIR, name.replace(".jpg", "_road.png")), 0)
        items.append({
            "name": name, "stem": name.replace(".jpg", ""), "img": img, "gt": gt,
            "condition": meta[name]["condition"], "note": meta[name]["note"],
            "split": "tuning" if name.endswith("_01.jpg") else "held out",
        })
    return items

def grey_for(img, filt="bilateral", space="HLS", strength=1.0):
    return P.road_channel(P.apply_filter(img, filt, strength), space)

# -----
def e1_colour(items):
    print("E1 colour representation")
    rows = []
    for it in items:
        row = [it["stem"], it["condition"]]
        for sp in P.COLOUR_SPACES:
            row.append(P.separability(it["img"], it["gt"], sp))
        rows.append(row)
    EV.write_csv(os.path.join(TAB, "e1_colour_separability.csv"),
                ["image", "condition"] + P.COLOUR_SPACES, rows)
    means = {sp: float(np.mean([r[2 + i] for r in rows]))
              for i, sp in enumerate(P.COLOUR_SPACES)}
    best = max(means, key=means.get)
    fig, ax = plt.subplots(figsize=(6.2, 3.2))
    ax.bar(list(means), [means[s] for s in means], color="#3b6ea5")
    ax.set_ylabel("mean Fisher separability")
    ax.set_title("Colour representation: road against everything else")
    fig.savefig(os.path.join(FIG, "fig02_colour.png")); plt.close(fig)
    print(" best representation: %s (%.2f)" % (best, means[best]))
    SUMMARY["E1"] = {"means": means, "best": best}
    return best

def e2_filters(items):
    """Four filters, scored the same way the edge operators are scored.

    The tempting measure is boundary recall divided by edge density, on the
    grounds that a good filter keeps the boundary and suppresses the rest. It
    is a trap, and an instructive one: a filter that blurs the image into
    porridge produces almost no edges at all, so the denominator collapses and
    the ratio goes up. Heavy Gaussian smoothing wins that measure while
    destroying nine tenths of the road boundary. Boundary F1 cannot be gamed

```

```

that way, because a filter that removes the boundary is punished by the
recall term.
"""
print("E2 filtering")
rows = []
for filt in P.FILTERS:
    rec, prec, dens, times = [], [], [], []
    for it in items:
        t0 = time.perf_counter()
        g = grey_for(it["img"], filt)
        em = E.detect_edges(g, "canny", 50, 150)
        times.append(time.perf_counter() - t0)
        roi = E.roi_mask(it["img"].shape)
        em = E.apply_roi(em)
        rec.append(E.boundary_recall(em, it["gt"]))
        prec.append(E.boundary_precision(em, it["gt"], mask=roi))
        dens.append(E.edge_density(em, roi))
    r, p = float(np.mean(rec)), float(np.mean(prec))
    f1 = 0.0 if (r + p) == 0 else 2 * r * p / (r + p)
    rows.append([filt, r, p, f1, float(np.mean(dens)), float(np.mean(times))])
EV.write_csv(os.path.join(TAB, "e2_filters.csv"),
             ["filter", "boundary_recall", "boundary_precision",
              "boundary_f1", "edge_density", "seconds_per_image"], rows)
best = max(rows, key=lambda r: r[3])[0]
fig, ax = plt.subplots(1, 2, figsize=(9.2, 3.2))
ax[0].bar([r[0] for r in rows], [r[1] for r in rows], color="#3b6ea5")
ax[0].set_ylabel("boundary recall")
ax[0].set_title("how much of the boundary survives")
ax[1].bar([r[0] for r in rows], [r[3] for r in rows], color="#a5643b")
ax[1].set_ylabel("boundary F1")
ax[1].set_title("recall and precision together")
for a in ax:
    a.tick_params(axis="x", rotation=15)
fig.savefig(os.path.join(FIG, "fig04_filters.png"))
plt.close(fig)
print("    best filter by boundary F1: %s" % best)
SUMMARY["E2"] = {"rows": rows, "best": best}
return best

def e3_edges(items, filt):
    """Canny against Sobel, scored by boundary F1 rather than recall alone.

    Recall on its own is a trap: an operator that marks every pixel as an edge
    scores a perfect recall and is useless. Boundary precision, the fraction of
    reported edges that actually lie on the road boundary, is what stops that,
    and the harmonic mean of the two is what selects the operating point.
    """
    print("E3 edge detection")
    rows = []
    for method in E.EDGE_METHODS:
        settings = ((30, 90), (50, 150), (80, 200), (120, 260))
        if method == "canny" else [(30,), (50,), (70,), (100,)]
        for setting in settings:
            rec, prec, dens = [], [], []
            for it in items:
                g = grey_for(it["img"], filt)
                roi = E.roi_mask(it["img"].shape)
                if method == "canny":
                    em = E.detect_edges(g, "canny", setting[0], setting[1])
                    label = "canny %d/%d" % setting
                else:
                    em = E.detect_edges(g, "sobel", sobel_thresh=setting[0])
                    label = "sobel %d" % setting[0]
                em = E.apply_roi(em)
                rec.append(E.boundary_recall(em, it["gt"]))
                prec.append(E.boundary_precision(em, it["gt"], mask=roi))
                dens.append(E.edge_density(em, roi))
            r, p = float(np.mean(rec)), float(np.mean(prec))
            f1 = 0.0 if (r + p) == 0 else 2 * r * p / (r + p)
            rows.append([label, method, r, p, f1, float(np.mean(dens))])
    EV.write_csv(os.path.join(TAB, "e3_edges.csv"),
                 ["setting", "method", "boundary_recall", "boundary_precision",
                  "boundary_f1", "edge_density"], rows)
    fig, ax = plt.subplots(figsize=(6.2, 3.6))
    for method, marker in (("canny", "o-"), ("sobel", "s-")):
        sel = [r for r in rows if r[1] == method]
        ax.plot([r[2] for r in sel], [r[3] for r in sel], marker, label=method)
    for r in sel:
        ax.annotate(r[0].split()[-1], (r[2], r[3]), fontsize=6,
                    textcoords="offset points", xytext=(3, 3))
    ax.set_xlabel("boundary recall"); ax.set_ylabel("boundary precision")
    ax.set_title("Edge operating curves: Canny against Sobel")
    ax.legend()
    fig.savefig(os.path.join(FIG, "fig05_edges.png")); plt.close(fig)
    best = max(rows, key=lambda r: r[4])
    print("    best operating point: %s (boundary F1 %.3f)" % (best[0], best[4]))
    SUMMARY["E3"] = {"rows": rows, "best": best[0]}
    return best[0]

def adaptive_roi_mask(img):
    """Derive the search region from a first pass segmentation.

    The fixed trapezoid encodes an assumption about where the camera points.
    An alternative is to let the image say where the road is: segment once,
    dilate the result generously, and search for boundaries only there. This
    costs one extra segmentation and removes the geometric assumption.
    """
    rough = S.segment(img, "threshold", k_sigma=3.0)

```

```

k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (41, 41))
return cv2.dilate(rough, k, iterations=1)

def e4_roi(items, filt):
    print("E4 region of interest")
    rows = []
    for label in ("whole frame", "fixed trapezoid", "adaptive from segmentation"):
        kept, errs, found = [], [], 0
        for it in items:
            g = grey_for(it["img"], filt)
            em = E.detect_edges(g, "canny", 50, 150)
            if label == "fixed trapezoid":
                em = E.apply_roi(em)
            elif label.startswith("adaptive"):
                em = cv2.bitwise_and(em, adaptive_roi_mask(it["img"]))
            res = L.detect_lanes(em, it["img"].shape)
            kept.append(len(res["segments"]))
            for side in ("left", "right"):
                if res[side] is not None:
                    found += 1
                    e = L.boundary_error(res[side], it["gt"], side)
                    if e is not None:
                        errs.append(e)
            rows.append([label, float(np.mean(kept)), found, 2 * len(items),
                        float(np.mean(errs)) if errs else float("nan"), len(errs)])
    EV.write_csv(os.path.join(TAB, "e4_roi.csv"),
                ["configuration", "mean_hough_segments", "boundaries_fitted",
                 "boundaries_possible", "mean_boundary_error_px",
                 "boundaries_scored"], rows)
    for r in rows:
        print(" %-27s %6.0f segments, %d/%d fitted, error %.1f px"
              % (r[0], r[1], r[2], r[3], r[4]))
    SUMMARY["E4"] = rows
    return rows

def e5_boundaries(items, filt, edge_cfg, seg_method, seg_kw):
    """Boundary identification: three strategies, and the Hough parameters.

    Strategy A fits straight lines to Hough segments taken from the edge map
    inside the region of interest, which is the textbook lane finder.
    Strategy B first gates those edges with the dilated road paint mask, so
    that only edges lying on white or yellow markings can vote.
    Strategy C ignores the edge map and reads the boundary straight off the
    segmented road region, taking the extreme road pixel in each row.
    """
    print("E5 boundary identification")

    sweep = []
    for thr in (20, 25, 40, 60):
        for mll in (25, 40, 70):
            errs, found = [], 0
            for it in items:
                em = E.apply_roi(edge_map_for(it["img"], filt, edge_cfg))
                res = L.detect_lanes(em, it["img"].shape,
                                    {"threshold": thr, "min_line_len": mll})
                for side in ("left", "right"):
                    if res[side] is not None:
                        found += 1
                        e = L.boundary_error(res[side], it["gt"], side)
                        if e is not None:
                            errs.append(e)
            sweep.append([thr, mll, found, 2 * len(items),
                        float(np.mean(errs)) if errs else float("nan")])
    EV.write_csv(os.path.join(TAB, "e5_hough_sweep.csv"),
                ["hough_threshold", "min_line_length", "boundaries_found",
                 "boundaries_possible", "mean_boundary_error_px"], sweep)
    ok = [r for r in sweep if not np.isnan(r[4]) and r[2] >= 2]
    best_h = min(ok, key=lambda r: r[4]) if ok else sweep[0]
    hough = {"threshold": int(best_h[0]), "min_line_len": int(best_h[1])}

    fig, ax = plt.subplots(figsize=(6.2, 3.4))
    for mll in (25, 40, 70):
        sel = [r for r in sweep if r[1] == mll]
        ax.plot([r[0] for r in sel], [r[4] for r in sel], "o-",
                label="min length %d" % mll)
    ax.set_xlabel("Hough accumulator threshold")
    ax.set_ylabel("mean boundary error (px)")
    ax.set_title("Lane boundary accuracy against the Hough threshold")
    ax.legend()
    fig.savefig(os.path.join(FIG, "fig06_hough.png"))
    plt.close(fig)

    rows = []
    for label in ("hough on ROI edges", "hough on paint edges",
                 "from segmented region"):
        errs, found, possible = [], 0, 2 * len(items)
        for it in items:
            em = E.apply_roi(edge_map_for(it["img"], filt, edge_cfg))
            if label == "hough on ROI edges":
                res = L.detect_lanes(em, it["img"].shape, hough)
            elif label == "hough on paint edges":
                res = L.detect_lanes(em, it["img"].shape, hough,
                                    restrict=L.paint_mask(it["img"]))
            else:
                res = L.boundaries_from_region(
                    S.segment(it["img"], seg_method, **seg_kw))
            for side in ("left", "right"):
                if res[side] is not None:

```

```

        found += 1
        e = L.boundary_error(res[side], it["gt"], side)
        if e is not None:
            errs.append(e)
        rows.append([label, found, possible, len(errs),
                     float(np.mean(errs)) if errs else float("nan")])
    EV.write_csv(os.path.join(TAB, "e5_boundary_strategies.csv"),
                 ["strategy", "boundaries_found", "boundaries_possible",
                  "boundaries_scored", "mean_boundary_error_px"], rows)

    for r in rows:
        txt = ("%1f px" % r[4]) if not np.isnan(r[4]) else "n/a"
        print("%-24s found %2d/%d error %s" % (r[0], r[1], r[2], txt))
        scored = [r for r in rows if not np.isnan(r[4])]
        best = min(scored, key=lambda r: r[4]) if scored else rows[0]
        print("    chosen: %s (Hough threshold=%d, min length=%d)"
              % (best[0], hough["threshold"], hough["min_line_len"]))
    SUMMARY["E5"] = {"sweep": sweep, "strategies": rows, "best": best[0],
                    "hough": hough}
    return hough, best[0]

def e6_segmentation(items):
    print("E6 segmentation")
    configs = []
    for ks in (2.0, 2.5, 3.0):
        configs.append(("threshold", {"k_sigma": ks, "threshold k=%1f" % ks}))
    for chi in (2.5, 3.0, 3.5):
        configs.append(("mahalanobis", {"chi": chi,
                                         "mahalanobis chi=%1f" % chi}))
    for k in (2, 3, 4, 5):
        configs.append(("kmeans", {"k": k, "kmeans K=%d" % k}))
    for tol in (8, 14, 20, 28):
        configs.append(("region_growing", {"tolerance": tol,
                                           "region growing tol=%d" % tol}))

    rows = []
    for method, kw, label in configs:
        per, times = [], []
        for it in items:
            t0 = time.perf_counter()
            m = S.segment(it["img"], method, **kw)
            times.append(time.perf_counter() - t0)
            per.append(EV.scores(m, it["gt"]))
        mean = EV.mean_scores(per)
        rows.append([label, method, mean["mean_iou"], mean["mean_precision"],
                     mean["mean_recall"], mean["mean_f1"], float(np.mean(times))])
    EV.write_csv(os.path.join(TAB, "e6_segmentation.csv"),
                 ["configuration", "method", "mean_iou", "mean_precision",
                  "mean_recall", "mean_f1", "seconds_per_image"], rows)

    best = max(rows, key=lambda r: r[2])
    fig, ax = plt.subplots(figsize=(8.6, 3.4))
    colours = {"threshold": "#3b6ea5", "mahalanobis": "#6b3ba5",
              "kmeans": "#a5643b", "region_growing": "#4f8a4f"}
    ax.bar([r[0] for r in rows], [r[2] for r in rows],
           color=[colours[r[1]] for r in rows])
    ax.set_ylabel("mean IoU"); ax.tick_params(axis="x", rotation=30)
    ax.set_title("Segmentation approaches and their parameters (tuning set)")
    fig.savefig(os.path.join(FIG, "fig07_segmentation.png")); plt.close(fig)
    print("    best: %s (IoU %.3f)" % (best[0], best[2]))
    SUMMARY["E6"] = {"rows": rows, "best": best}
    return best

def edge_from_label(label):
    """Turn the winning label from E3 back into an edge configuration."""
    if label.startswith("canny"):
        lo, hi = label.split()[1].split("/")
        return {"method": "canny", "low": int(lo), "high": int(hi)}
    return {"method": "sobel", "sobel_thresh": int(label.split()[1])}

def edge_map_for(img, filt, edge_cfg):
    """Filtering and edge detection, exactly as the pipeline does it."""
    g = grey_for(img, filt)
    if edge_cfg["method"] == "canny":
        return E.detect_edges(g, "canny", edge_cfg["low"], edge_cfg["high"])
    return E.detect_edges(g, "sobel", sobel_thresh=edge_cfg["sobel_thresh"])

def seg_params(seg_best):
    """Turn the winning label from E6 back into keyword arguments."""
    method, label = seg_best[1], seg_best[0]
    value = label.split("=")[1]
    if method == "threshold":
        return {"k_sigma": float(value)}
    if method == "mahalanobis":
        return {"chi": float(value)}
    if method == "kmeans":
        return {"k": int(value)}
    return {"tolerance": int(value)}

def build_pipeline_config(filt, edge_label, hough, seg_best, boundary):
    method = seg_best[1]
    kw = seg_params(seg_best)
    edge = edge_from_label(edge_label)
    return {"filter": filt, "colour_space": "HLS", "edge": edge,
            "hough": hough, "boundary_strategy": boundary,
            "segmentation": {"method": method, "params": kw}}

def run_pipeline(img, cfg):
    """Road image -> filtering -> edges -> ROI -> boundaries -> segmentation."""

```

```

        filtered = P.apply_filter(img, cfg["filter"])
        grey = P.road_channel(filtered, cfg["colour_space"])
        e = cfg["edge"]
        if e["method"] == "canny":
            em = E.detect_edges(grey, "canny", e["low"], e["high"])
        else:
            em = E.detect_edges(grey, "sobel", sobel_thresh=e["sobel_thresh"])
        em_roi = E.apply_roi(em)
        road = S.segment(filtered, cfg["segmentation"]["method"],
                        **cfg["segmentation"]["params"])

        hough_lanes = L.detect_lanes(em_roi, img.shape, cfg["hough"])
        paint = L.paint_mask(img)
        paint_lanes = L.detect_lanes(em_roi, img.shape, cfg["hough"],
                                    restrict=paint)
        region_lanes = L.boundaries_from_region(road)
        chosen = {"hough on ROI edges": hough_lanes,
                 "hough on paint edges": paint_lanes,
                 "from segmented region": region_lanes}[cfg["boundary_strategy"]]

    return {"filtered": filtered, "grey": grey, "edges": em,
            "edges_roi": em_roi, "paint": paint, "lanes": chosen,
            "hough_lanes": hough_lanes, "region_lanes": region_lanes,
            "road": road}

def e7_final(items, cfg):
    print("E7 assembled pipeline")
    rows, per_cond = [], {}
    for it in items:
        t0 = time.perf_counter()
        out = run_pipeline(it["img"], cfg)
        secs = time.perf_counter() - t0
        sc = EV.scores(out["road"], it["gt"])
        el = L.boundary_error(out["lanes"]["left"], it["gt"], "left")
        er = L.boundary_error(out["lanes"]["right"], it["gt"], "right")
        berr = [v for v in (el, er) if v is not None]
        rows.append([it["stem"], it["condition"], it["split"], sc["iou"],
                    sc["precision"], sc["recall"], sc["f1"],
                    float(np.mean(berr)) if berr else float("nan"),
                    len(berr), secs])
        c = per_cond.setdefault(it["condition"], [])
        c.append(sc)
        cv2.imwrite(os.path.join(OVL, "final%s.jpg" % it["stem"]),
                    S.overlay_result(it["img"], out["road"], out["lanes"]),
                    [cv2.IMWRITE_JPEG_QUALITY, 92])
    EV.write_csv(os.path.join(TAB, "e7_final_per_image.csv"),
                ["image", "condition", "split", "iou", "precision", "recall",
                 "f1", "boundary_error_px", "boundaries_found",
                 "seconds"], rows)

    cond_rows = []
    for cond, lst in sorted(per_cond.items()):
        m = EV.mean_scores(lst)
        cond_rows.append([CONDITION.get(cond, cond), len(lst), m["mean_iou"],
                        m["mean_precision"], m["mean_recall"], m["mean_f1"]])
    EV.write_csv(os.path.join(TAB, "e7_by_condition.csv"),
                ["condition", "images", "mean_iou", "mean_precision",
                 "mean_recall", "mean_f1"], cond_rows)

    split_rows = []
    for split in ("tuning", "held out"):
        sel = [r for r in rows if r[2] == split]
        if not sel:
            continue
        split_rows.append([split, len(sel),
                        float(np.mean([r[3] for r in sel])),
                        float(np.mean([r[4] for r in sel])),
                        float(np.mean([r[5] for r in sel])),
                        float(np.mean([r[6] for r in sel])),
                        (float(np.nanmean([r[7] for r in sel]))
                         if np.any(~np.isnan([r[7] for r in sel]))
                         else float("nan"))])
    EV.write_csv(os.path.join(TAB, "e7_by_split.csv"),
                ["split", "images", "mean_iou", "mean_precision", "mean_recall",
                 "mean_f1", "mean_boundary_error_px", split_rows)

    fig, ax = plt.subplots(figsize=(7.6, 3.4))
    ax.bar([r[0] for r in cond_rows], [r[2] for r in cond_rows], color="#3b6ea5")
    ax.set_ylabel("mean IoU"); ax.set_ylim(0, 1)
    ax.tick_params(axis="x", rotation=20)
    ax.set_title("Road region IoU by imaging condition")
    fig.savefig(os.path.join(FIG, "fig09_by_condition.png")); plt.close(fig)

    for r in split_rows:
        print("%-9s IoU %.3f P %.3f R %.3f F1 %.3f boundary %.1f px"
              % (r[0], r[2], r[3], r[4], r[5], r[6]))
    SUMMARY["E7"] = {"per_image": rows, "by_condition": cond_rows,
                    "by_split": split_rows, "config": cfg}
    return rows, cond_rows, split_rows

# -----
def figures(items, cfg):
    print("figures")
    # dataset montage
    fig, axes = plt.subplots(3, 4, figsize=(12.4, 6.6))
    for a, it in zip(axes.ravel(), items):
        vis = it["img"].copy()
        sel = it["gt"] &gt; 0

```

```

vis[sel] = (np.array((0, 210, 0)) * 0.34 + vis[sel] * 0.66).astype(np.uint8)
a.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
a.set_title("%s\n%s" % (it["stem"], CONDITION.get(it["condition"], "")),
            fontsize=7)
a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.suptitle("The dataset with the manual road annotation in green", fontsize=10)
fig.savefig(os.path.join(FIG, "fig01_dataset.png")); plt.close(fig)

# pipeline stages
it = next(x for x in items if x["stem"] == "daylight_02")
out = run_pipeline(it["img"], cfg)
roi_vis = it["img"].copy()
cv2.polylines(roi_vis, E.roi_polygon(it["img"].shape), True, (0, 180, 255), 3)
panels = [
    (cv2.cvtColor(it["img"], cv2.COLOR_BGR2RGB), "1. road image"),
    (cv2.cvtColor(out["filtered"], cv2.COLOR_BGR2RGB), "2. filtering"),
    (out["edges"], "3. edge detection"),
    (cv2.cvtColor(roi_vis, cv2.COLOR_BGR2RGB), "4. region selection"),
    (out["edges_roi"], "5. edges inside the ROI"),
    (cv2.cvtColor(L.draw_segments(it["img"], out["lanes"]), cv2.COLOR_BGR2RGB),
     "6. Hough segments"),
    (out["road"], "7. segmentation"),
    (cv2.cvtColor(S.overlay_result(it["img"], out["road"], out["lanes"]),
     cv2.COLOR_BGR2RGB), "8. final visualisation"),
]
fig, axes = plt.subplots(2, 4, figsize=(13.0, 4.6))
for a, (im, title) in zip(axes.ravel(), panels):
    a.imshow(im, cmap=None if im.ndim == 3 else "gray")
    a.set_title(title, fontsize=8)
    a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.savefig(os.path.join(FIG, "fig03_pipeline.png")); plt.close(fig)

# segmentation methods side by side on two images
fig, axes = plt.subplots(2, 1 + len(S.METHODS), figsize=(15.5, 4.8))
for row, stem in enumerate(["shadow_01", "worn_02"]):
    it = next(x for x in items if x["stem"] == stem)
    axes[row][0].imshow(cv2.cvtColor(it["img"], cv2.COLOR_BGR2RGB))
    axes[row][0].set_title("%s" % stem, fontsize=8)
    for col, method in enumerate(S.METHODS, start=1):
        m = S.segment(it["img"], method)
        sc = EV.scores(m, it["gt"])
        axes[row][col].imshow(cv2.cvtColor(
            S.overlay_result(it["img"], m), cv2.COLOR_BGR2RGB))
        axes[row][col].set_title("%s, IoU %.2f" % (method, sc["iou"]), fontsize=8)
for a in axes.ravel():
    a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.savefig(os.path.join(FIG, "fig08_methods.png")); plt.close(fig)

# failure montage: the four worst images by IoU
scored = []
for it in items:
    out = run_pipeline(it["img"], cfg)
    scored.append((EV.scores(out["road"], it["gt"])["iou"], it, out))
scored.sort(key=lambda t: t[0])
fig, axes = plt.subplots(1, 4, figsize=(13.0, 2.9))
for a, (iou, it, out) in zip(axes, scored[:4]):
    a.imshow(cv2.cvtColor(S.overlay_result(it["img"], out["road"], out["lanes"]),
     cv2.COLOR_BGR2RGB))
    a.set_title("%s (%s)\nIoU %.2f" % (it["stem"],
     CONDITION.get(it["condition"], ""), iou),
     fontsize=8)
    a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.suptitle("Where the system fails", fontsize=10)
fig.savefig(os.path.join(FIG, "fig10_failures.png")); plt.close(fig)

def main():
    items = load_dataset()
    tuning = [it for it in items if it["split"] == "tuning"]
    print("%d images, %d tuning, %d held out\n"
          % (len(items), len(tuning), len(items) - len(tuning)))
    SUMMARY["dataset"] = {"images": len(items), "tuning": len(tuning),
                          "held_out": len(items) - len(tuning)}

    best_space = e1_colour(items)
    best_filter = e2_filters(tuning)
    best_edge = e3_edges(tuning, best_filter)
    e4_roi(tuning, best_filter)
    seg_best = e6_segmentation(tuning)
    hough, boundary = e5_boundaries(tuning, best_filter,
                                   edge_from_label(best_edge),
                                   seg_best[1], seg_params(seg_best))

    cfg = build_pipeline_config(best_filter, best_edge, hough, seg_best,
                               boundary)
    EV.write_json(os.path.join(RES, "pipeline_config.json"), cfg)
    print("\npipeline: %s\n" % json.dumps(cfg))

    e7_final(items, cfg)
    figures(items, cfg)

    SUMMARY["colour_note"] = best_space
    EV.write_json(os.path.join(RES, "summary.json"), SUMMARY)
    print("\nall results written to results/")

if __name__ == "__main__":
    main()

```

src/make_report.py

```
"""
make_report.py
Builds the report PDF from the measured results in results/.

Every number quoted in the document is read from the CSV files written by
run_experiments.py, so the report cannot drift away from the experiments.

python src/make_report.py

from __future__ import annotations

import csv
import json
import os

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.lib.utils import ImageReader
from reportlab.platypus import (BaseDocTemplate, Frame, Image, KeepTogether,
                                NextPageTemplate, PageBreak, PageTemplate,
                                Paragraph, Preformatted, Spacer, Table,
                                TableStyle)

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RES = os.path.join(ROOT, "results")
FIG = os.path.join(RES, "figures")
TAB = os.path.join(RES, "tables")
OUT = os.path.join(ROOT, "report", "Road_and_Lane_Boundary_Detection.pdf")

NAME = "Nii Yartey Gidiglo"
PROJECT = "Road and lane boundary detection"

PAGE_W, PAGE_H = A4
MARGIN = 2.0 * cm
BODY_W = PAGE_W - 2 * MARGIN

ss = getSampleStyleSheet()
BODY = ParagraphStyle("body", parent=ss["BodyText"], fontName="Helvetica",
                      fontSize=9.6, leading=13.6, alignment=TA_JUSTIFY,
                      spaceAfter=6)
H1 = ParagraphStyle("h1", parent=ss["Heading1"], fontName="Helvetica-Bold",
                    fontSize=14, leading=17, spaceBefore=14, spaceAfter=7,
                    textColor=colors.HexColor("#12305c"))
H2 = ParagraphStyle("h2", parent=ss["Heading2"], fontName="Helvetica-Bold",
                    fontSize=11.2, leading=14, spaceBefore=10, spaceAfter=5,
                    textColor=colors.HexColor("#1d4d8f"))
H3 = ParagraphStyle("h3", parent=ss["Heading3"], fontName="Helvetica-Bold",
                    fontSize=10, leading=13, spaceBefore=8, spaceAfter=4)
CAP = ParagraphStyle("cap", parent=BODY, fontSize=8.2, leading=10.5,
                    alignment=TA_CENTER, textColor=colors.HexColor("#444444"),
                    spaceBefore=3, spaceAfter=10)
EQ = ParagraphStyle("eq", parent=BODY, fontName="Courier", fontSize=9,
                    leading=12.5, alignment=TA_CENTER, spaceBefore=5, spaceAfter=7)
BULLET = ParagraphStyle("bullet", parent=BODY, leftIndent=14, bulletIndent=4,
                        spaceAfter=3)
CODE = ParagraphStyle("code", parent=ss["Code"], fontName="Courier",
                      fontSize=6.0, leading=7.2, spaceAfter=0, spaceBefore=0)
TITLE = ParagraphStyle("title", parent=BODY, fontName="Helvetica-Bold",
                       fontSize=19, leading=24, alignment=TA_CENTER,
                       spaceAfter=8, textColor=colors.HexColor("#12305c"))
SUB = ParagraphStyle("sub", parent=BODY, fontSize=11.5, leading=16,
                    alignment=TA_CENTER, spaceAfter=4)

story = []
FIGN = [0]
TABN = [0]

def p(text, style=BODY):
    story.append(Paragraph(text, style))

def h1(t): story.append(Paragraph(t, H1))
def h2(t): story.append(Paragraph(t, H2))
def h3(t): story.append(Paragraph(t, H3))
def eq(t): story.append(Paragraph(t, EQ))

def bullets(items):
    for it in items:
        story.append(Paragraph(it, BULLET, bulletText="•"))
    story.append(Spacer(1, 4))

def figure(fname, caption, width=BODY_W):
    path = os.path.join(FIG, fname)
    if not os.path.exists(path):
        return
    iw, ih = ImageReader(path).getSize()
    w = min(width, BODY_W)
    h = w * ih / float(iw)
    if h > 15.0 * cm:
        h = 15.0 * cm
        w = h * iw / float(ih)
```

```

FIGN[0] += 1
story.append(KeepTogether([Image(path, w, h),
                             Paragraph("Figure %d. %s" % (FIGN[0], caption), CAP)]))

def read_table(name):
    with open(os.path.join(TAB, name), encoding="utf-8") as fh:
        rows = list(csv.reader(fh))
    return rows[0], rows[1:]

def table(header, rows, caption, widths=None, fontsize=7.4, left_first=True):
    TABN[0] += 1
    head_style = ParagraphStyle("th", parent=BODY, fontSize=fontsize,
                                leading=fontsize + 2, alignment=TA_CENTER,
                                textColor=colors.white, spaceAfter=0)
    cell = ParagraphStyle("td", parent=BODY, fontSize=fontsize,
                           leading=fontsize + 2, alignment=TA_CENTER, spaceAfter=0)
    cell_l = ParagraphStyle("tdl", parent=cell, alignment=0)
    data = [[Paragraph("<b>%s</b>" % h.replace("_", " "), head_style) for h in header]]
    for r in rows:
        data.append([Paragraph(str(v), cell_l if (i == 0 and left_first) else cell)
                      for i, v in enumerate(r)])
    t = Table(data, colWidths=widths, repeatRows=1, hAlign="CENTER")
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1d4d8f")),
        ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#9aa7b5")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("ROWBACKGROUND", (0, 1), (-1, -1), [colors.white, colors.HexColor("#eef2f7")]),
        ("LEFTPADDING", (0, 0), (-1, -1), 3), ("RIGHTPADDING", (0, 0), (-1, -1), 3),
        ("TOPPADDING", (0, 0), (-1, -1), 2.5), ("BOTTOMPADDING", (0, 0), (-1, -1), 2.5),
    ]))
    story.append(KeepTogether([t, Paragraph("Table %d. %s" % (TABN[0], caption), CAP)]))

def num(x, nd=3):
    """Format a number, rendering a missing measurement as n/a."""
    try:
        v = float(x)
    except (TypeError, ValueError):
        return str(x)
    if v != v:  # NaN, meaning nothing could be measured
        return "n/a"
    return "%. " + str(nd) + "f" % v

def esc(t):
    return t.replace("&", "&&").replace("<", "&lt;").replace(">", "&gt;")

def decorate(canvas, doc):
    canvas.saveState()
    canvas.setFont("Helvetica", 7.5)
    canvas.setFillColor(colors.HexColor("#5a6673"))
    canvas.drawString(MARGIN, 1.15 * cm, PROJECT)
    canvas.drawRightString(PAGE_W - MARGIN, 1.15 * cm, "%s, page %d" % (NAME, doc.page))
    canvas.setStrokeColor(colors.HexColor("#c7d0da"))
    canvas.line(MARGIN, 1.45 * cm, PAGE_W - MARGIN, 1.45 * cm)
    canvas.restoreState()

def blank(canvas, doc):
    pass

def build():
    doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=MARGIN, rightMargin=MARGIN,
                          topMargin=1.7 * cm, bottomMargin=2.0 * cm,
                          title="Road and Lane Boundary Detection", author=NAME)
    frame = Frame(MARGIN, 2.0 * cm, BODY_W, PAGE_H - 3.7 * cm, id="body")
    doc.addPageTemplates([PageTemplate(id="cover", frames=[frame], onPage=blank),
                          PageTemplate(id="main", frames=[frame], onPage=decorate)])
    doc.build(story)

def main():
    summary = json.load(open(os.path.join(RES, "summary.json"), encoding="utf-8"))
    cfg = json.load(open(os.path.join(RES, "pipeline_config.json"), encoding="utf-8"))
    sources = json.load(open(os.path.join(ROOT, "data", "sources.json"), encoding="utf-8"))

    h_e1, r_e1 = read_table("e1_colour_separability.csv")
    h_e2, r_e2 = read_table("e2_filters.csv")
    h_e3, r_e3 = read_table("e3_edges.csv")
    h_e4, r_e4 = read_table("e4_roi.csv")
    h_e5, r_e5 = read_table("e5_boundary_strategies.csv")
    h_e5s, r_e5s = read_table("e5_hough_sweep.csv")
    h_e6, r_e6 = read_table("e6_segmentation.csv")
    h_e7i, r_e7i = read_table("e7_final_per_image.csv")
    h_e7c, r_e7c = read_table("e7_by_condition.csv")
    h_e7s, r_e7s = read_table("e7_by_split.csv")

    best_colour = summary["E1"]["best"]
    colour_means = summary["E1"]["means"]
    best_filter = summary["E2"]["best"]
    e2_none = next(r for r in r_e2 if r[0] == "none")
    e2_gauss = next(r for r in r_e2 if r[0] == "gaussian")
    e2_bil = next(r for r in r_e2 if r[0] == "bilateral")
    best_e6 = max(r_e6, key=lambda r: float(r[2]))
    best_thr = max((r for r in r_e6 if r[1] == "threshold"), key=lambda r: float(r[2]))
    best_km = max((r for r in r_e6 if r[1] == "kmeans"), key=lambda r: float(r[2]))

```

```

best_rg = max((r for r in r_e6 if r[1] == "region_growing"), key=lambda r: float(r[2]))
e5_region = next(r for r in r_e5 if r[0] == "from segmented region")
e5_hough = next(r for r in r_e5 if r[0] == "hough on ROI edges")
e5_paint = next(r for r in r_e5 if r[0] == "hough on paint edges")
tune = next(r for r in r_e7s if r[0] == "tuning")
held = next(r for r in r_e7s if r[0] == "held out")
worst_cond = min(r_e7c, key=lambda r: float(r[2]))
best_cond = max(r_e7c, key=lambda r: float(r[2]))

# ----- cover -----
story.append(Spacer(1, 2.6 * cm))
p("COMPUTER VISION PROJECT", SUB)
story.append(Spacer(1, 1.0 * cm))
p("Automated Road and Lane Boundary Detection<br>and Scene Segmentation", TITLE)
story.append(Spacer(1, 0.5 * cm))
p("Colour representation, filtering, edge detection, region selection, "
"segmentation and boundary identification, evaluated against hand traced "
"ground truth", SUB)
story.append(Spacer(1, 1.4 * cm))
info = Table(["Author", NAME],
             ["Repository", "github.com/Qwamzz/Computer-Vision-Projects"],
             ["Dataset", "%d real road photographs, six imaging conditions"
              % len(sources)],
             ["Ground truth", "hand traced drivable region for every image"],
             ["Implementation", "Python 3 and OpenCV, no learned model"]],
             colwidths=[4.2 * cm, 10.5 * cm])
info.setStyle(TableStyle([
    ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#9aa7b5")),
    ("BACKGROUND", (0, 0), (0, -1), colors.HexColor("#eef2f7")),
    ("FONTNAME", (0, 0), (0, -1), "Helvetica-Bold"),
    ("FONTSIZE", (0, 0), (-1, -1), 9),
    ("TOPPADDING", (0, 0), (-1, -1), 5), ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
]))
info.hAlign = "CENTER"
story.append(info)
story.append(Spacer(1, 1.2 * cm))
p("<br>Statement on the imagery.<br>Every photograph used in this project "
"was taken by someone else and published on Wikimedia Commons under a "
"free licence. The licence and author of each file are listed in Section "
"2.1 and in data/SOURCES.md. No image was produced by an image generation "
"model.",
ParagraphStyle("note", parent=BODY, fontSize=8.6, leading=12,
borderPadding=6, backColor=colors.HexColor("#f4f7fb")))
story.append(NextPageTemplate("main"))
story.append(PageBreak())

# ----- 1 -----
h1("1. Problem definition and objectives")
p("The task is to analyse a road scene and report two things: where the "
"drivable road surface is, and where its boundaries run. Both are wanted "
"from classical computer vision alone, with no learned model anywhere in "
"the pipeline.")
p("The system implements the full progression end to end:")
eq("road image -&gt; filtering -&gt; edge detection -&gt; region selection "
"-&gt; segmentation -&gt; boundary identification -&gt; visualisation")
p("Every stage offers a genuine choice, and this report settles each one by "
"measurement rather than by assertion:")
bullets([
    "which colour representation separates road from everything else,",
    "which filter suppresses noise without destroying the road boundary,",
    "whether Canny or Sobel is the better edge operator here,",
    "how much a region of interest helps, and what its assumption costs,",
    "which of four segmentation rules recovers the drivable region best,",
    "and whether boundaries are better found from edges or from the "
    "segmented region.",
])
p("The last of those turned out to be the most interesting question in the "
"project, and the answer is not the textbook one. It is reported in "
"Section 5.5 and discussed in Section 6.")

# ----- 2 -----
h1("2. Dataset and ground truth")
h2("2.1 The photographs")
p("Twelve real photographs of real roads were collected from Wikimedia "
"Commons, two for each of the six conditions the question asks the system "
"to be tested under. Using photographs taken by other people, for their "
"own purposes, with no common camera, mounting or exposure, is a harder "
"test than a dashcam sequence and a more honest one: nothing about the "
"geometry is shared between images, so no hidden constant can be tuned.")
table(["file", "condition", "what it tests", "licence"],
      [[s["file"], s["condition"].replace("_", " "), s["note"][:74],
        s["licence"]] for s in sources],
      ["The dataset. Every file is a real photograph under a free licence; "
"authors are listed in data/SOURCES.md.",
widths=[2.9 * cm, 2.6 * cm, 8.6 * cm, 2.9 * cm], fontsize=6.6])
figure("fig01_dataset.png",
      "The twelve photographs with the hand traced drivable region in "
      "green. The six conditions are two each of good daylight, shadows, "
      "poor illumination, curved roads, worn or absent markings, and "
      "occluding traffic.")
h2("2.2 Ground truth")
p("The drivable surface of every image was traced by hand as a polygon, "
"read off the photograph against a printed coordinate grid, and then "
"checked by drawing the resulting mask back over the photograph and "
"correcting whatever was wrong. Several images went round that loop more "
"than once. The vertices are stored in src/annotate.py rather than in a "
"binary mask, so anyone can see exactly what was claimed to be road and "
"can re-derive the masks from scratch.")
p("Two decisions in the annotation are worth stating because they change ")

```

```

"what the numbers mean. First, the annotated region is the sealed "
"carriageway, bounded by the painted edge lines where they exist and by "
"the change of surface where they do not. Second, anything standing on "
"that surface, a vehicle or an animal, is cut out of the mask, so the "
"ground truth is the &lt;i>visible&/i> drivable surface. A segmentation "
"method cannot reasonably be asked to report road where a bus is parked, "
"and scoring it as though it should would make the occlusion condition "
"meaningless.")

h2("2.3 Protocol")
p("The twelve images are split in half. The six ending in _01 are the "
"tuning set: every parameter in Sections 5.1 to 5.6 was chosen on those "
"and only those. The six ending in _02 were held out and run once, at the "
"end, with the parameters already fixed. Both halves contain one image "
"from each condition, so the held out half is not the easier half. The "
"gap between the two, reported in Section 5.7, is the honest measure of "
"how much of the tuning was real and how much was fitting to six "
"photographs.")

# ----- 3 -----
h1("3. Methodology")
figure("fig03_pipeline.png",
"The pipeline on one held out image, stage by stage: the input, the "
"filtered image, the raw edge map, the region of interest, the edges "
"that survive it, the Hough segments split into left and right, the "
"segmented road region, and the final overlay.")

h2("3.1 Colour representation")
p("Road surfaces here are grey asphalt or orange laterite, so they have no "
"colour in common. What they share is a &lt;i>lack&/i> of one: against the "
"vegetation, sky and painted buildings around them they are weakly "
"saturated and mid range in lightness. HLS puts exactly those two "
"quantities on their own axes, and its lightness channel is also where "
"white and yellow paint stands out most strongly. Lab is the alternative, "
"with approximately perceptually uniform a and b axes that suit "
"clustering. RGB is kept as the baseline that mixes brightness and chroma "
"together and therefore moves under every change of illumination.")

h2("3.2 Filtering")
p("A road boundary is a step edge that must survive; the texture of "
"chippings, gravel and dappled tree shade is noise that should not. "
"Gaussian smoothing cannot tell the two apart. A median filter removes "
"impulsive speckle while leaving step edges intact. A bilateral filter "
"weights by intensity difference as well as by distance, so it smooths "
"within a region and stops at the boundary between regions. All three "
"are compared against no filtering at all in Section 5.2.")

h2("3.3 Edge detection")
p("Sobel estimates the gradient with a small separable kernel and "
"thresholds its magnitude. Canny adds the three steps that make an edge "
"detector usable: smoothing, non maximum suppression across the gradient "
"direction so edges come out one pixel wide, and hysteresis thresholding "
"so a weak edge survives when it connects to a strong one. Hysteresis is "
"the reason Canny can hold on to a faded lane marking that a single "
"threshold on Sobel magnitude either loses or buries in noise.")

h2("3.4 Region selection")
p("Almost everything above the horizon is irrelevant, and it is also where "
"the strongest edges live: trees, buildings, poles and the skyline. Two "
"region selection strategies are compared. The fixed trapezoid covers the "
"carriageway in front of the camera and assumes the camera points along "
"the road. The adaptive region is derived from a first pass segmentation "
"and then dilated, which drops that assumption at the cost of one extra "
"segmentation. A horizon cut is also applied inside the segmentation "
"stage, because a surprising amount of sky is close enough to grey "
"asphalt in colour to be accepted by any of the rules below.")

h2("3.5 Segmentation")
p("Four rules are implemented. All four take a colour sample from a small "
"trapezoid immediately in front of the camera, which is the one place a "
"forward facing photograph is almost certain to be road, and all four "
"share the same morphological cleanup and the same final step of keeping "
"the connected component that reaches the bottom of the frame, since the "
"drivable region must by definition reach the vehicle.")
bullets([
"&lt;b>Adaptive thresholding.&/b> A per channel acceptance band in HLS, "
"set at the sample mean plus or minus k standard deviations.",
"&lt;b>Mahalanobis thresholding.&/b> The same idea with the correlation "
"between channels put back in: the full covariance of the sample is "
"estimated and a pixel is accepted when its Mahalanobis distance from "
"the sample mean is small.",
"&lt;b>K means clustering.&/b> The image is clustered in Lab and the "
"cluster that dominates the near field sample is taken to be road. No "
"threshold is chosen at all, but K must be.",
"&lt;b>Region growing.&/b> A flood fill from seeds in the near field, "
"which is the only one of the four that enforces spatial connectivity, "
"so a grey roof on the far side of a hedge cannot be labelled road.",
])
eq("Mahalanobis: sqrt( (x - mu)^T C^-1 (x - mu) ) &lt;= chi")

h2("3.6 Boundary identification")
p("Three strategies are compared. The first is the textbook lane finder: "
"the probabilistic Hough transform turns edge pixels into line segments, "
"segments are sorted into a left and a right boundary by the sign of "
"their slope, nearly horizontal segments are discarded because they "
"belong to shadows and to the backs of vehicles, and each side is reduced "
"to one line by least squares weighted by segment length. The second "
"gates the same edges with a mask of white and yellow road paint, so only "
"edges lying on markings can vote. The third ignores edges entirely and "
"reads the boundary off the segmented region, taking the extreme road ")

```



```

    "Canny and Sobel across their thresholds, measured against the true "
    "road boundary inside the region of interest.",
    widths=[3.0 * cm, 2.4 * cm, 3.0 * cm, 3.2 * cm, 2.6 * cm, 2.6 * cm],
    fontsize=7.2)
figure("fig05_edges.png",
    "Operating curves. Each point is one threshold setting; the "
    "annotation is the threshold.", 11 * cm)
p("Both operators score poorly in absolute terms, and that is the finding "
    "rather than a defect of the measurement. Boundary precision never "
    "exceeds about %s, because the overwhelming majority of edges inside the "
    "region of interest are real edges of things that are not the road "
    "boundary: the texture of the surface, the shadows of roadside trees, "
    "vegetation, buildings and the backs of vehicles. An edge detector cannot "
    "distinguish them, because at the level of a local gradient they are not "
    "different. This is the quantitative reason why the boundary stage in "
    "Section 5.5 does better when it stops relying on edges."
    % num(max(float(r[3]) for r in r_e3), 2))
p("Between the two, Sobel holds a slight edge on this measure at the "
    "operating points tested, mainly because Canny after the smoothing stage "
    "leaves very little behind: at 120/260 it finds almost nothing at all. "
    "The choice matters far less than the table suggests, since neither "
    "operator supports a reliable boundary on its own.")

h2("5.4 Region selection")
table(h_e4, [[r[0]] + [num(v, 1) if i >= 1 else r[1] for i, v in enumerate(r[1:])]
    for r in r_e4],
    "What the region of interest removes, and what it does for the "
    "accuracy of the boundaries fitted inside it.",
    widths=[5.2 * cm, 3.2 * cm, 2.6 * cm, 2.8 * cm, 3.2 * cm],
    fontsize=7.2)
p("Restricting the search cuts the number of Hough segments by roughly a "
    "factor of two and improves the accuracy of the boundaries that are "
    "fitted. The adaptive region, derived from a first pass segmentation "
    "rather than from an assumption about where the camera points, is the "
    "better of the two: it reaches %s pixels of mean boundary error against "
    "%s for the fixed trapezoid and %s for searching the whole frame. That "
    "ordering is worth noting because the fixed trapezoid is the standard "
    "choice, and it is standard because dashcam frames share a geometry that "
    "this dataset deliberately does not."
    % (num(r_e4[2][4], 1), num(r_e4[1][4], 1), num(r_e4[0][4], 1)))

h2("5.5 Boundary identification")
table(h_e5, [[r[0]] + [num(v, 1) if i == 3 else r[i + 1]
    for i, v in enumerate(r[1:])] for r in r_e5],
    "The three boundary strategies on the tuning set. Error is the mean "
    "horizontal distance from the true road edge, in pixels, on images "
    "1280 pixels wide.",
    widths=[5.0 * cm, 3.0 * cm, 3.2 * cm, 3.0 * cm, 3.2 * cm], fontsize=7.2)
figure("fig06_hough.png",
    "Mean boundary error against the Hough accumulator threshold, for "
    "three minimum segment lengths.", 11 * cm)
p("This is the result that reshaped the project. Fitting straight lines to "
    "Hough segments, the textbook method, gives a mean boundary error of %s "
    "pixels. Gating those edges with the road paint mask does not rescue it "
    "(%s pixels) and finds fewer boundaries, because half the dataset has no "
    "usable paint at all. Reading the boundary off the segmented region "
    "instead gives %s pixels, three to four times better, on images 1280 "
    "pixels wide."
    % (num(e5_hough[4], 1), num(e5_paint[4], 1), num(e5_region[4], 1)))
p("The reason is visible in Section 5.3. The Hough transform is an "
    "excellent way to find lines in a set of points that mostly lie on lines. "
    "Here the edge map is dominated by points that lie on other things, and "
    "the transform has no way to know which is which; a few long segments on "
    "a hedge or a shadow are enough to drag a least squares fit a long way. "
    "The segmented region has already used colour to decide what is road, so "
    "its boundary is a boundary of the right object by construction. The "
    "safeguards in Section 3.6 keep the Hough path honest rather than "
    "accurate: they convert many of its worst fits into no answer at all.")

h2("5.6 Segmentation")
table(h_e6, [[r[0], r[1].replace("_", " ")] + [num(v) for v in r[2:]] for r in r_e6],
    "All four segmentation rules across their parameters, on the tuning "
    "set.",
    widths=[4.0 * cm, 2.8 * cm, 2.2 * cm, 2.4 * cm, 2.2 * cm, 2.0 * cm,
    2.6 * cm], fontsize=6.8)
figure("fig07_segmentation.png",
    "Mean IoU for every configuration tested. Colour groups the "
    "configurations by method.")
figure("fig08_methods.png",
    "The four rules on a shadowed road and on an unsealed one, with the "
    "IoU each achieves.")
p("The best configuration is %s at an IoU of %s, ahead of the best per "
    "channel threshold (%s), the best K means (%s) and the best region "
    "growing (%s). The margin over the per channel threshold is the "
    "interesting part, because the two rules differ only in whether the "
    "correlation between colour channels is used. Treating the channels "
    "independently accepts an axis aligned box in colour space; the road "
    "colours actually form a tilted, elongated cloud, so a box wide enough to "
    "contain them also admits a great deal of vegetation and sky. Putting the "
    "covariance back in fixes that, and it is the single largest accuracy "
    "gain in the project."
    % (best_e6[0], num(best_e6[2]), num(best_thr[2]), num(best_km[2]),
    num(best_rg[2])))
p("The other two rules fail in characteristic ways. K means is unstable in "
    "K: with two clusters it splits the image into bright and dark and calls "
    "half the scene road, and with five it fragments the road surface across "
    "several clusters. Region growing is the most precise of all at a tight "
    "tolerance (precision %s at tolerance 8) and the least complete (recall "
    "%s), because a shadow band across the carriageway stops the fill dead; "
    "opening the tolerance far enough to cross the shadow also opens it ")

```

```

"enough to leak into the verge."
% (num(next(r for r in r_e6 if r[0].endswith("tol=8"))[3]),
  num(next(r for r in r_e6 if r[0].endswith("tol=8"))[4]))

h2("5.7 The assembled system")
p("The configuration selected on the tuning half was: %s filtering, the HLS "
  "lightness channel, %s edges, a fixed trapezoid region of interest, %s "
  "segmentation, and boundaries %s. It was then run once on the held out "
  "half."
  % (cfg["filter"], cfg["edge"]["method"],
    cfg["segmentation"]["method"], cfg["boundary_strategy"]))
table(h_e7s, [[r[0], r[1]] + [num(v) for v in r[2:6]] + [num(r[6], 1)]
  for r in r_e7s],
  "Tuning half against held out half, with all parameters fixed before "
  "the held out half was run.",
  widths=[2.6 * cm, 2.0 * cm, 2.4 * cm, 2.6 * cm, 2.4 * cm, 2.2 * cm,
    3.4 * cm], fontsize=7.2)
p("The system reaches an IoU of %s on the tuning half and %s on the held "
  "out half, with F1 falling from %s to %s and mean boundary error rising "
  "from %s to %s pixels. A drop of that size across the split is worth "
  "stating plainly: with six images to tune on, some of the apparent "
  "accuracy was fitted to those six photographs. The held out figure is the "
  "one to believe."
  % (num(tune[2]), num(held[2]), num(tune[5]), num(held[5]),
    num(tune[6], 1), num(held[6], 1)))
table(h_e7i, [[r[0], r[1].replace("-", " "), r[2]] + [num(v) for v in r[3:7]]
  + [num(r[7], 1), r[8], num(r[9], 2)] for r in r_e7i],
  "Every image individually under the final configuration. A boundary "
  "error of n/a means no boundary could be scored, either because none "
  "was found or because the road leaves the frame on both sides.",
  widths=[2.6 * cm, 2.4 * cm, 1.8 * cm, 1.5 * cm, 1.7 * cm, 1.5 * cm,
    1.4 * cm, 2.0 * cm, 1.6 * cm, 1.4 * cm], fontsize=6.4)
table(h_e7c, [[r[0], r[1]] + [num(v) for v in r[2:]] for r in r_e7c],
  "Performance by imaging condition, over all twelve images.",
  widths=[4.6 * cm, 2.2 * cm, 2.4 * cm, 2.6 * cm, 2.4 * cm, 2.2 * cm],
  fontsize=7.2)
figure("fig09_by_condition.png",
  "Road region IoU by imaging condition.", 12.5 * cm)
p("The ordering by condition is close to the ordering a person would "
  "predict. The system is strongest on %s (IoU %s) and weakest on %s (IoU "
  "%s). The occlusion case is the most interesting of the failures: "
  "precision stays high at %s while recall collapses to %s, which is "
  "exactly the signature of a method that is finding real road and only "
  "real road, but cannot see most of it because vehicles are standing on "
  "it."
  % (best_cond[0], num(best_cond[2]), worst_cond[0], num(worst_cond[2]),
    num(next(r for r in r_e7c if "occluding" in r[0])[3]),
    num(next(r for r in r_e7c if "occluding" in r[0])[4])))
figure("fig10_failures.png",
  "The four weakest results. Green is the segmented road region, red "
  "the fitted boundaries.")

# ----- 6 -----
h1("6. Critical discussion: where the system fails and why")
p("The question asks for the situations in which the classical algorithms "
  "break down, and for the reasons. Each of the following is visible in the "
  "per condition table or in the failure figure.")

h3("Poor illumination and haze")
p("This is the weakest condition, at an IoU of %s. Haze compresses the "
  "whole scene towards a narrow band of grey, which is precisely the band "
  "the road occupies. The near field sample then has a small variance, the "
  "Mahalanobis ellipsoid it defines is correspondingly narrow along every "
  "axis, and distant vegetation of nearly the same grey falls inside it "
  "while the far part of the road, which is hazier still, falls outside. "
  "The result is high recall with poor precision. No amount of parameter "
  "tuning fixes this, because the information that separates road from "
  "verge has been removed by the atmosphere before the camera sees it."
  % num(worst_cond[2]))

h3("Occluding traffic")
p("A vehicle standing on the carriageway removes the road from view, and "
  "there is nothing classical or otherwise to be done about the pixels it "
  "covers. What the system does do correctly is refuse to guess: precision "
  "on this condition is %s. The failure that matters is a different one. "
  "Where a queue of vehicles hides the road edge, the connected component "
  "that reaches the camera is bounded by the vehicles rather than by the "
  "road, so the fitted boundary follows the traffic. A human driver infers "
  "the road continues behind the bus; the system has no mechanism for that "
  "inference."
  % num(next(r for r in r_e7c if "occluding" in r[0])[3]))

h3("Shadows")
p("Dappled shade from roadside trees is the classic failure of any method "
  "that decides by colour, and it defeats region growing completely: a "
  "shadow band across the carriageway is, in pixel terms, a wall. The "
  "Mahalanobis rule survives it better because a shadowed patch of asphalt "
  "is displaced from the sample mean mostly along the lightness axis, which "
  "the covariance already knows is the direction of greatest variation, so "
  "the ellipsoid is elongated in exactly that direction. That is why the "
  "shadow condition scores %s rather than the 0.23 that region growing "
  "reaches on the same images."
  % num(next(r for r in r_e7c if r[0] == "shadows")[2]))

h3("Curves")
p("The straight line model is wrong on a curve by construction, and the "
  "boundary error shows it: the curved images are among the worst for "
  "boundary accuracy even though their region IoU is high (%s). The region "
  "is found correctly and then described badly. A second order fit in the "
  "image plane, or a fit in a bird's eye view obtained by an inverse "

```

```

"perspective mapping, would address this directly and is the first thing "
"worth adding."
% num(next(r for r in r_e7c if "curved" in r[0])[2]))

h3("Worn and absent markings")
p("The unsealed roads score well on region IoU (%s), which is initially "
"surprising, until one notices that laterite against green vegetation is "
"a far larger colour contrast than grey asphalt against a grey verge. "
"what fails on these images is the paint based boundary strategy, which "
"has nothing to find: the boundary here is a change of material, not a "
"painted line. This is the clearest case in the dataset for preferring a "
"region derived boundary over an edge derived one."
% num(next(r for r in r_e7c if "worn" in r[0])[2]))

h3("The near field assumption")
p("Every segmentation rule here trusts that the bottom centre of the frame "
"is road. It is the assumption that makes a per image threshold possible "
"at all, and it holds for a forward facing camera on a vehicle. It does "
"not hold for a photograph taken from the verge, and one image in this "
"dataset comes close to breaking it. A system deployed on arbitrary "
"photographs would need to verify the assumption rather than rely on it.")

h2("6.1 Limitations")
bullets([
    "Twelve images with two per condition is a small evaluation. Condition "
    "level numbers rest on two photographs each and should be read as "
    "indicative.",
    "The ground truth is hand traced by one annotator with no second "
    "opinion, and the boundary of a road into an unsealed verge is "
    "genuinely ambiguous to within a few pixels.",
    "Every image is a single frame. A video sequence would allow the "
    "boundary to be tracked over time, which is the single largest "
    "practical improvement available and is not attempted here.",
    "The system reports a drivable region and its outer boundaries, not "
    "individual lanes. Finding the lane within a road needs the paint, and "
    "half of this dataset has none.",
])
h2("6.2 What would help most")
bullets([
    "An inverse perspective mapping to a bird's eye view, in which lane "
    "boundaries are parallel and a quadratic fit is well conditioned.",
    "Temporal tracking across frames, so that a boundary hidden by a "
    "vehicle in one frame is carried forward from the last.",
    "A shadow invariant colour representation, which would attack the "
    "shadow and haze failures at their source rather than downstream.",
    "Verifying the near field assumption instead of trusting it, for "
    "example by checking that the sample region is texturally uniform.",
])

# ----- 7 -----
h1("7. Conclusion")
p("A complete classical road and lane boundary system was built and "
"measured end to end, from colour representation through filtering, edge "
"detection, region selection and segmentation to boundary identification "
"and visualisation. On six held out photographs that played no part in "
"tuning, it recovers the drivable region with an IoU of %s and an F1 of "
"%s, and places the road boundaries to within %s pixels on images 1280 "
"pixels wide."
% (num(held[2]), num(held[5]), num(held[6], 1)))
p("Three findings are worth carrying away. First, the measure used to "
"select a stage matters as much as the stage: two natural looking "
"criteria, recall divided by edge density for the filter and for the edge "
"operator, both select a configuration that destroys the boundary, "
"because their denominators collapse faster than their numerators. "
"Second, putting the correlation between colour channels back into a "
"simple threshold, which is all the Mahalanobis rule does, was worth more "
"accuracy than any other single change. Third, and least expected, the "
"textbook Hough based lane finder was beaten by a wide margin by reading "
"the boundary off the segmented region, because on ordinary road "
"photographs the edge map is dominated by edges of things that are not "
"the road, and the Hough transform has no way to tell them apart.")

# ----- references -----
h1("References")
for i, ref in enumerate([
    "Canny, J. (1986). A computational approach to edge detection. IEEE "
    "Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679 to 698.",
    "Duda, R. O. and Hart, P. E. (1972). Use of the Hough transformation to "
    "detect lines and curves in pictures. Communications of the ACM, 15(1), "
    "11 to 15.",
    "Otsu, N. (1979). A threshold selection method from gray level "
    "histograms. IEEE Transactions on Systems, Man and Cybernetics, 9(1), "
    "62 to 66.",
    "Tomasi, C. and Manduchi, R. (1998). Bilateral filtering for gray and "
    "colour images. Proceedings of ICCV, pages 839 to 846.",
    "Aly, M. (2008). Real time detection of lane markers in urban streets. "
    "IEEE Intelligent Vehicles Symposium, pages 7 to 12.",
    "Mahalanobis, P. C. (1936). On the generalised distance in statistics. "
    "Proceedings of the National Institute of Sciences of India, 2(1), 49 to 55.",
    "Szeliski, R. (2022). Computer Vision: Algorithms and Applications, "
    "second edition. Springer.",
    "Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.",
])
1):
    p("[%d] %s" % (i, ref),
        ParagraphStyle("ref", parent=BODY, leftIndent=16, firstLineIndent=-16,
            spaceAfter=4))

# ----- appendix -----
story.append(PageBreak())
h1("Appendix A. Complete source code")

```

```

p("The listing below is the complete source of the project, in the order in "
  "which the modules are run. The same files are in the src directory of "
  "the repository.")
for fname in ["download_data.py", "annotate.py", "preprocessing.py",
              "edges.py", "lanes.py", "segmentation.py", "evaluate.py",
              "run_experiments.py", "make_report.py"]:
    path = os.path.join(ROOT, "src", fname)
    if not os.path.exists(path):
        continue
    h2("src/" + fname)
    with open(path, encoding="utf-8") as fh:
        lines = fh.read().split("\n")
    chunk = []
    for line in lines:
        chunk.append(esc(line.rstrip()[:112]))
        if len(chunk) == 46:
            story.append(Preformatted("\n".join(chunk), CODE))
            chunk = []
    if chunk:
        story.append(Preformatted("\n".join(chunk), CODE))
    story.append(Spacer(1, 6))

os.makedirs(os.path.dirname(OUT), exist_ok=True)
build()
print("wrote", OUT)

if __name__ == "__main__":
    main()

```